0.1 CATLOG

0.1 CATLOG

1 一、pandas相关

1.1 多sheet页保存

1.1.1 同时保存多个sheet

1.1.2 追加新sheet页

1.2 相关性矩阵（皮尔逊）

1.2.1 代码

1.2.2 输出样式

1.3 读取hive表数据

1.4 单变量分析（二分类）

1.4.1 代码

1.4.2 输出样式

1.5 其他小方法

1.5.1 随机打乱+恢复索引

1.5.2 随机数种子

1.5.3 重命名列名

2 二、连续变量分箱（二分类）

2.1 等距离分箱

2.1.1 等距离分箱_只输出分箱结果

2.1.2 等距离分箱_输出每条数据对应分箱

2.2 等频率分箱

2.2.1 等频率分箱_只输出分箱结果

2.2.2 等频率分箱_输出每条数据对应分箱

3 三、模型相关

3.1 决策树可视化

3.1.1 graphviz 库

3.1.2 dtreeviz库

3.2 随机森林填补缺失值（回归）

3.3 模型保存

3.3.1 joblib库

3.3.2 pickle

4 四、日期相关

4.1 FuncDateReFormat

4.2 FuncLastDay

4.3 FuncDateAdd

4.4 FuncGenDateList

4.5 对系统当前时间操作

4.6 综合

5 五、风控、机器学习相关

5.1 woe分箱_iv值计算

5.2 vif-方差膨胀因子

5.3 KS值和KS曲线
5.4 lift-提升度
5.5 psi+csi
5.5.1 PSI-群体稳定性指标(连续型)
5.5.2 PSI-群体稳定性指标(离散型)
5.5.3 CSI-特征稳定性指标
5.6 KL散度
5.6.1 信息熵
5.6.2 联合熵
5.6.3 交叉熵
5.6.4 KL散度（相对熵）
5.6.5 交叉熵误差函数
6 补充

1 一、pandas相关

1.1 多sheet页保存

1.1.1 同时保存多个sheet

1.1.1.1 代码

- 代码位置

> /Users/mengzhichao/Desktop/文件/16、python代码/Sample_Code/2、method/7、excel同时保存多个sheet.py

- 同时保存多个sheet页，但是如果该excel文件之前存在，会覆盖其中已经有的sheet页。
- 核心代码1：直接存新的文件

```
with pd.ExcelWriter('./save/excel_file.xlsx', mode='w') as writer:
    data.to_excel(writer, sheet_name='table1')
    data.to_excel(writer, sheet_name='table2')
    data.to_excel(writer, sheet_name='table3')
```

- 核心代码2：在原有的文件上追加

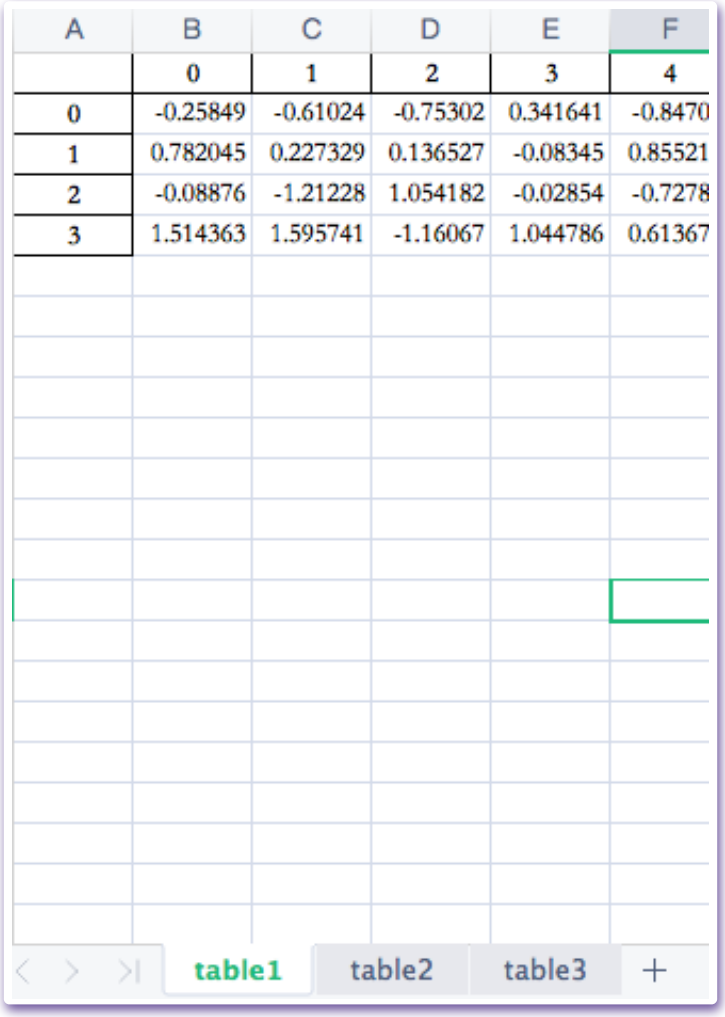
```
with pd.ExcelWriter('./save/excel_file.xlsx',
mode='a',engine='openpyxl') as writer:
    data.to_excel(writer,sheet_name='table4')
```

- 完整代码：

```
# -*- coding: utf-8 -*-

"""
**************************************************
@author:   Ying
@software: PyCharm
@file: 7、excel同时保存多个sheet.py
@time: 2021-07-21 14:16
**************************************************
"""
import pandas as pd
import numpy as np

data = pd.DataFrame(np.random.randn(20).reshape(4, 5))

with pd.ExcelWriter('./save/excel_file.xlsx', mode='w') as writer:
    data.to_excel(writer, sheet_name='table1')
    data.to_excel(writer, sheet_name='table2')
    data.to_excel(writer, sheet_name='table3')
```

```python

with pd.ExcelWriter('./save/excel_file.xlsx',mode='a',engine='openpyxl') as writer:
    data.to_excel(writer,sheet_name='table4')
```

1.1.1.2 输出样式



	0	1	2	3	4
0	-0.25849	-0.61024	-0.75302	0.341641	-0.8470
1	0.782045	0.227329	0.136527	-0.08345	0.85521
2	-0.08876	-1.21228	1.054182	-0.02854	-0.7278
3	1.514363	1.595741	-1.16067	1.044786	0.61367

table1 table2 table3

- 备注：用 `with` 打开，就不用再 `save` 和 `close` 了

1.1.2 追加新sheet页

1.1.2.1 代码

- 代码位置：

> /Users/mengzhichao/Desktop/文件/16、python代码/Sample_Code/2、method/8、excel追加sheet页.py

- 可以追加新的sheet页到已经存在若干sheet页的excel文件中，且不覆盖先前的sheet页。
- 核心代码：

```
with pd.ExcelWriter('./save/excel_file.xlsx', engine='openpyxl') as writer:
    book = load_workbook('./save/excel_file.xlsx')
    writer.book = book
    data = pd.DataFrame(np.random.randn(10, 5))
    data.to_excel(writer, sheet_name='new_sheet')
```

- 完整代码：

```
# -*- coding: utf-8 -*-

"""
**************************************************
@author:   Ying
@software: PyCharm
@file: 8、excel追加sheet页.py
@time: 2021-07-21 14:30
**************************************************
"""

from openpyxl import load_workbook
import pandas as pd
import numpy as np

with pd.ExcelWriter('./save/excel_file.xlsx', engine='openpyxl') as writer:
    book = load_workbook('./save/excel_file.xlsx')
    writer.book = book
    data = pd.DataFrame(np.random.randn(10, 5))
    data.to_excel(writer, sheet_name='new_sheet')

```

1.1.2.2 输出样式

	0	1	2	3	4
0	-0.2476	-0.92924	1.034294	-0.36965	-1.30732
1	-0.6131	0.059915	0.642713	1.296062	2.439517
2	0.683041	-0.86488	0.088811	-1.06592	-0.2495
3	-0.01307	-0.23334	0.546928	-0.99579	-2.21779
4	1.407091	2.350817	0.139605	-0.413	1.647816
5	-0.33315	1.02464	0.96234	-0.50941	-0.58928
6	0.488263	-0.77336	-0.08363	0.082929	-0.87586
7	0.190829	0.448013	-0.55171	-1.7238	-1.4867
8	2.986102	2.007263	-1.02147	0.447464	-0.01576
9	-0.52797	-0.1125	0.914049	-1.4262	0.298423

table1 table2 table3 new_sheet

1.2 相关性矩阵（皮尔逊）

1.2.1 代码

- 代码位置：

> /Users/mengzhichao/Desktop/文件/16、python代码/Sample_Code/2、method/1、皮尔逊相关性矩阵.py

- 核心代码：

```
def plot_corr_matrix(corrcoef):
    fig = plt.figure(figsize=(10, 10))
    # Blues,Greens,Reds,Greys,Purples,Oranges,BuGn,Purples.......
    ax = sns.heatmap(corrcoef
                     , cmap=plt.cm.Blues
                     , square=True
                     , vmax=1
                     , vmin=0
                     , annot=True
                     , fmt='.2f'
```

```
                        )

    ax.set_title('指标相关性矩阵')  # 标题设置
    plt.savefig('./save/皮尔逊相关性.png',dpi=1000)
    plt.show()
```

- 完整代码：

```
# -*- coding: utf-8 -*-
"""
**************************************************
@author:   Ying
@software: PyCharm
@file: 1、皮尔逊相关性矩阵.py
@time: 2020/11/23 15:22
**************************************************
"""

import pandas as pd
import matplotlib
from matplotlib import pyplot as plt
import seaborn as sns
from sklearn.datasets import load_breast_cancer

plt.rcParams["font.family"] = 'Arial Unicode MS'
matplotlib.rcParams.update({'font.size': 2})  # 设置全体字体大小


def plot_corr_matrix(corrcoef):
    """
    绘制 相关性矩阵 热力图，保存相关性矩阵图形
    :param corrcoef: 相关性系数
    :return:
    """
    fig = plt.figure(figsize=(10, 10))
    # Blues,Greens,Reds,Greys,Purples,Oranges,BuGn.......
    ax = sns.heatmap(corrcoef
                     , cmap=plt.cm.Blues
                     , square=True
                     # , linewidths=0.05
                     , vmax=1
                     , vmin=0
                     , annot=True
                     , fmt='.2f'
                     # , annot_kws={'size': 10, 'weight': 'bold'}
                     )
    # 热力图参数设置（相关系数矩阵，颜色，每个值间隔等）
    # ticks = numpy.arange(0,16,1) #生成0-16，步长为1
    # plt.xticks(np.arange(len(columns_name)) + 0.5, columns_name) # 横坐标标注点
    # plt.yticks(np.arange(len(columns_name)) + 0.5, columns_name) # 纵坐标标注点
```

```
    # ax.set_xticks(ticks) #生成刻度
    # ax.set_yticks(ticks)++++++
```

1.3 读取hive表数据

- 代码位置

> /Users/mengzhichao/Desktop/文件/16、python代码/Sample_Code/2、method/21、hive表读取.py

- 读取hive表数据

```python
# -*- coding: utf-8 -*-

"""
**************************************************
@author:   Ying
@software: PyCharm
@file: 21、hive表读取.py
@time: 2022-08-31 10:39
**************************************************
"""

import pandas as pd
import numpy as np
from tqdm import tqdm

NP = np

fp = "./"  # 从hive中读取的数据表存放地址
fp_desc = ''  # 数据表desc文件存放地址
table_name = ''  # 数据表名称


def read_table_from_hive(fp, fp_desc, table_name, chunk_flag=False, col_list=None, subset=None):
    """
    pandas 读取hive表文件

    :param fp:          hive表存放地址
    :param fp_desc:     hive表对应的desc 文件存放地址
    :param table_name:  hive表名称
    :param chunk_flag:  大规模数据集是否按块进行读取,默认正常读取
    :param col_list:    指定读取的列，默认为所有列
    :param subset:      是否按照指定维度去重，默认不去重，如果去重的话，返回两个DataFrame，原始的和去重后的
    :return:
    """
    # 创建宽表dtype字典，指定连续型为float32类型
    dtype_data = pd.read_csv(fp_desc,sep='\t')
    dtype_data = dtype_data.iloc[:-5, :]
    dtype_data = dtype_data.applymap(lambda x: x.strip())
```

```python
    dtype_data['col_name'] = dtype_data['col_name'].apply(lambda x: table_name + '.' + x)
    dtype_data['data_type'] = dtype_data['data_type'].apply(lambda x: 'str' if x == "string" else 'float32')
    dtype_dict = dict(dtype_data[['col_name', 'data_type']].values) # 数据表dtype字典

    # 指定读取的列表
    if col_list:  # 传入指定的列表
        table_col_list = [table_name + '.' + x for x in col_list]
    else:  # 默认读取全部
        table_col_list = dtype_data.col_name.tolist()

    # 读取数据表
    if chunk_flag:
        df_chunk = pd.read_csv(fp
                               , sep='\t'  # 换行符
                               , usecols=table_col_list  # 指定读取列表
                               , dtype=dtype_dict  # 指定读取数据类型
                               , chunksize=100000  # 块读取条数
                               )
        res_chunk = []
        for chunk in tqdm(df_chunk):
            res_chunk.append(chunk)
        df_origin = pd.concat(res_chunk)
    else:
        df_origin = pd.read_csv(fp
                                , sep='\t'  # 换行符
                                , usecols=table_col_list  # 指定读取列表
                                , dtype=dtype_dict  # 指定读取数据类型
                                )

    df_origin.columns = [i.split('.')[1] for i in df_origin.columns]  # 去除hive表前缀
    # df_origin.drop_duplicates(inplace=True)  # 去重
    # df_origin.index = range(df_origin.shape[0])

    if subset:
        df = df_origin.drop_duplicates(subset=subset)
        df.index = range(df.shape[0])

        return df_origin, df
    else:
        return df_origin

```

1.4 单变量分析（二分类）

1.4.1 代码

- 代码位置

> /Users/mengzhichao/Desktop/文件/16、python代码/Sample_Code/2、method/13、单变量分析.py

- 核心代码

```python
def box_outlier(data_series):
    """

    :param data_series: 某列数据
    :return: iqr、上下限、异常值数
    """
    q1 = data_series.describe()['25%']  # 下四分位数
    q3 = data_series.describe()['75%']  # 上四分位数
    iqr = q3 - q1  # 四分位全距

    up_limit = q3 + 1.5 * iqr  # 上限
    down_limit = q1 - 1.5 * iqr  # 下限

    outlier_num = data_series[(data_series > up_limit) | (data_series < down_limit)].shape[0]

    return iqr, up_limit, down_limit, outlier_num


def univariate_analysis(data, feature_list, label):
    """

    :param data: 数据
    :param feature_list: 特征列表（连续型）
    :param label: 目标变量（y）
    :return:
    """
    # 样本总数
    sample_num = data.shape[0]

    # 描述性统计数据
    describe_data = data.describe([0.01, 0.05, 0.1, 0.25, 0.5, 0.75, 0.9, 0.99]).T.reset_index()
    describe_data.columns = ['字段名', '计数_数据量', '均值', '标准差', '最小值',
                             '1%', '5%', '10%', '第一四分位数(q1)', '中位数',
```

```
                                '第三四分位数(q3)', '90%', '99%', '最大值']
    describe_data['极差'] = describe_data['最大值'] - describe_data['最小值']

    # 样本缺失数、样本缺失率
    missing_data = pd.DataFrame(data.isnull().sum())
    missing_data.columns = ['样本缺失数']
    missing_data['样本缺失率'] = missing_data['样本缺失数'] / sample_num

    result_data = pd.merge(describe_data, missing_data, left_on='字段名', right_index=True)

    # 众数，众数统计次数，
    # k-s检验
    # s-w检验
    # 峰度，偏度
    # 众数异常值数(箱线图)、异常值率(箱线图)
    # 零值数、零值率、正样本数、正样本缺失数、正样本缺失率、正样本零值数、正样本零值率、
    #                 负样本数、负样本缺失数、负样本缺失率、负样本零值数、负样本零值率
    statistical_value_dict = {}
    statistical_value_dict['字段名'] = []

    # 众数，众数统计次数
    statistical_value_dict['众数'] = []
    statistical_value_dict['众数统计次数'] = []

    # k-s检验
    # 输出结果中第一个为统计量，第二个为P值（注：统计量越接近0就越表明数据和标准正态分布拟合的越好，
    # 如果P值大于显著性水平，通常是0.05，接受原假设，则判断样本的总体服从正态分布
    statistical_value_dict['K-S检验(正态_双尾)_统计量'] = []
    statistical_value_dict['K-S检验(正态_双尾)_P值'] = []

    # s-w检验
    statistical_value_dict['S-W检验_统计量'] = []
    statistical_value_dict['S-W检验_P值'] = []

    # 峰度，偏度
    statistical_value_dict['峰度'] = []
    statistical_value_dict['偏度'] = []

    # 箱线图
    statistical_value_dict['iqr(箱线图)'] = []
    statistical_value_dict['上限(箱线图)'] = []
    statistical_value_dict['下限(箱线图)'] = []
    statistical_value_dict['异常值数(箱线图)'] = []
    statistical_value_dict['异常值率(箱线图)'] = []
```

```python

    statistical_value_dict['零值数'] = []
    statistical_value_dict['零值率'] = []
    statistical_value_dict['正样本数'] = []
    statistical_value_dict['正样本率'] = []
    statistical_value_dict['正样本缺失数'] = []
    statistical_value_dict['正样本缺失率'] = []
    statistical_value_dict['正样本零值数'] = []
    statistical_value_dict['正样本零值率'] = []
    statistical_value_dict['负样本数'] = []
    statistical_value_dict['负样本率'] = []
    statistical_value_dict['负样本缺失数'] = []
    statistical_value_dict['负样本缺失率'] = []
    statistical_value_dict['负样本零值数'] = []
    statistical_value_dict['负样本零值率'] = []
    statistical_value_dict['方差'] = []

    for feature in feature_list:
        statistical_value_dict['字段名'].append(feature)

        # 众数、众数统计次数
        mode_num = stats.mode(data[feature])[0][0]
        mode_appear_num = stats.mode(data[feature])[1][0]
        statistical_value_dict['众数'].append(mode_num)
        statistical_value_dict['众数统计次数'].append(mode_appear_num)

        # ks检验
        statistical_value_dict['K-S检验(正态_双尾)_统计量'].append(list(stats.kstest(data[feature]
         , cdf='norm'  # 检验方法（累计密度函数）
         , alternative='two_sided'  # 双尾
         ))[0])
        statistical_value_dict['K-S检验(正态_双尾)_P值'].append(list(stats.kstest(data[feature]
        , cdf='norm'
        , alternative='two_sided'  # 双尾


        ))[1])
        # s-w检验（小样本）
        statistical_value_dict['S-W检验_统计量'].append(list(stats.shapiro(data[feature]))[0])
        statistical_value_dict['S-W检验_P值'].append(list(stats.shapiro(data[feature]))[1])
        # 峰度偏度
```

```
        statistical_value_dict['峰度'].append(data[feature].kurt())
        statistical_value_dict['偏度'].append(data[feature].skew())

        # 箱线图
        iqr, up_limit, down_limit, outlier_num = box_outlier(data[feature])
        outlier_rate = outlier_num / sample_num
        statistical_value_dict['iqr(箱线图)'].append(iqr)
        statistical_value_dict['上限(箱线图)'].append(up_limit)
        statistical_value_dict['下限(箱线图)'].append(down_limit)
        statistical_value_dict['异常值数(箱线图)'].append(outlier_num)
        statistical_value_dict['异常值率(箱线图)'].append(outlier_rate)

        zero_num = data[data[feature] == 0].shape[0]
        zero_rate = zero_num / sample_num
        statistical_value_dict['零值数'].append(zero_num)
        statistical_value_dict['零值率'].append(zero_rate)

        positive_num = data[data[label] == 1].shape[0]
        positive_rate = positive_num / sample_num
        positive_missing_num = data[data[label] == 1][feature].isnull().sum()
        positive_missing_rate = positive_missing_num / positive_num
        positive_zero_num = data[(data[label] == 1) & (data[feature] == 0)][feature].shape[0]
        positive_zero_rate = positive_zero_num / positive_num
        statistical_value_dict['正样本数'].append(positive_num)
        statistical_value_dict['正样本率'].append(positive_rate)
        statistical_value_dict['正样本缺失数'].append(positive_missing_num)
        statistical_value_dict['正样本缺失率'].append(positive_missing_rate)
        statistical_value_dict['正样本零值数'].append(positive_zero_num)
        statistical_value_dict['正样本零值率'].append(positive_zero_rate)

        negative_num = data[data[label] == 0].shape[0]
        negative_rate = negative_num / sample_num
        negative_missing_num = data[data[label] == 0][feature].isnull().sum()
        negative_missing_rate = negative_missing_num / negative_num
        negative_zero_num = data[(data[label] == 0) & (data[feature] == 0)][feature].shape[0]
        negative_zero_rate = negative_zero_num / negative_num
        statistical_value_dict['负样本数'].append(negative_num)
```

```
            statistical_value_dict['负样本率'].append(negative_rate)
            statistical_value_dict['负样本缺失数'].append(negative_missing_num)
            statistical_value_dict['负样本缺失率'].append(negative_missing_rate)
            statistical_value_dict['负样本零值数'].append(negative_zero_num)
            statistical_value_dict['负样本零值率'].append(negative_zero_rate)

            var_value = np.var(data[feature])
            statistical_value_dict['方差'].append(var_value)

        statistical_value = pd.DataFrame(statistical_value_dict)

        result_data = pd.merge(result_data, statistical_value, on='字段名', how='left')
        return result_data

```

- 所有代码

```
# -*- coding: utf-8 -*-

"""
**************************************************
@author:   Ying
@software: PyCharm
@file: 13、单变量分析.py
@time: 2021-10-14 14:13
**************************************************
"""

import pandas as pd
from sklearn.datasets import load_breast_cancer
import numpy as np
from scipy import stats

wine = load_breast_cancer()
data = pd.DataFrame(wine.data, columns=wine.feature_names)
data['y'] = wine.target


def box_outlier(data_series):
    """

    :param data_series: 某列数据
    :return: iqr、上下限、异常值数
    """
    q1 = data_series.describe()['25%']  # 下四分位数
    q3 = data_series.describe()['75%']  # 上四分位数
    iqr = q3 - q1  # 四分位全距
```

```python

    up_limit = q3 + 1.5 * iqr  # 上限
    down_limit = q1 - 1.5 * iqr  # 下限

    outlier_num = data_series[(data_series > up_limit) | (data_series < down_limit)].shape[0]

    return iqr, up_limit, down_limit, outlier_num


def univariate_analysis(data, feature_list, label):
    """

    :param data: 数据
    :param feature_list: 特征列表（连续型）
    :param label: 目标变量（y）
    :return:
    """
    # 样本总数
    sample_num = data.shape[0]

    # 描述性统计数据
    describe_data = data.describe([0.01, 0.05, 0.1, 0.25, 0.5, 0.75, 0.9, 0.99]).T.reset_index()
    describe_data.columns = ['字段名', '计数_数据量', '均值', '标准差', '最小值',
                             '1%', '5%', '10%', '第一四分位数(q1)', '中位数',
                             '第三四分位数(q3)', '90%', '99%', '最大值']
    describe_data['极差'] = describe_data['最大值'] - describe_data['最小值']

    # 样本缺失数、样本缺失率
    missing_data = pd.DataFrame(data.isnull().sum())
    missing_data.columns = ['样本缺失数']
    missing_data['样本缺失率'] = missing_data['样本缺失数'] / sample_num

    result_data = pd.merge(describe_data, missing_data, left_on='字段名', right_index=True)

    # 众数，众数统计次数，
    # k-s检验
    # s-w检验
    # 峰度，偏度
    # 众数异常值数(箱线图)、异常值率(箱线图)
    # 零值数、零值率、正样本数、正样本缺失数、正样本缺失率、正样本零值数、正样本零值率、
    #               负样本数、负样本缺失数、负样本缺失率、负样本零值数、负样本零值率
    statistical_value_dict = {}
```

```python
    statistical_value_dict['字段名'] = []

    # 众数，众数统计次数
    statistical_value_dict['众数'] = []
    statistical_value_dict['众数统计次数'] = []

    # k-s检验
    # 输出结果中第一个为统计量，第二个为P值（注：统计量越接近0就越表明数据和标准正态分布拟合的越好，
    # 如果P值大于显著性水平，通常是0.05，接受原假设，则判断样本的总体服从正态分布
    statistical_value_dict['K-S检验(正态_双尾)_统计量'] = []
    statistical_value_dict['K-S检验(正态_双尾)_P值'] = []

    # s-w检验
    statistical_value_dict['S-W检验_统计量'] = []
    statistical_value_dict['S-W检验_P值'] = []

    # 峰度，偏度
    statistical_value_dict['峰度'] = []
    statistical_value_dict['偏度'] = []

    # 箱线图
    statistical_value_dict['iqr(箱线图)'] = []
    statistical_value_dict['上限(箱线图)'] = []
    statistical_value_dict['下限(箱线图)'] = []
    statistical_value_dict['异常值数(箱线图)'] = []
    statistical_value_dict['异常值率(箱线图)'] = []

    statistical_value_dict['零值数'] = []
    statistical_value_dict['零值率'] = []
    statistical_value_dict['正样本数'] = []
    statistical_value_dict['正样本率'] = []
    statistical_value_dict['正样本缺失数'] = []
    statistical_value_dict['正样本缺失率'] = []
    statistical_value_dict['正样本零值数'] = []
    statistical_value_dict['正样本零值率'] = []
    statistical_value_dict['负样本数'] = []
    statistical_value_dict['负样本率'] = []
    statistical_value_dict['负样本缺失数'] = []
    statistical_value_dict['负样本缺失率'] = []
    statistical_value_dict['负样本零值数'] = []
    statistical_value_dict['负样本零值率'] = []
    statistical_value_dict['方差'] = []

    for feature in feature_list:
        statistical_value_dict['字段名'].append(feature)

        # 众数、众数统计次数
        mode_num = stats.mode(data[feature])[0][0]
        mode_appear_num = stats.mode(data[feature])[1][0]
        statistical_value_dict['众数'].append(mode_num)
```

```
        statistical_value_dict['众数统计次数'].append(mode_appear_num)

        # ks检验
        statistical_value_dict['K-S检验(正态_双尾)_统计量'].append(list(stats.kstest(data[feature]
         , cdf='norm'  # 检验方法（累计密度函数）
         , alternative='two_sided'  # 双尾
         ))[0])
        statistical_value_dict['K-S检验(正态_双尾)_P值'].append(list(stats.kstest(data[feature]
        , cdf='norm'
        , alternative='two_sided'  # 双尾


        ))[1])
        # s-w检验（小样本）
        statistical_value_dict['S-W检验_统计量'].append(list(stats.shapiro(data[feature]))[0])
        statistical_value_dict['S-W检验_P值'].append(list(stats.shapiro(data[feature]))[1])
        # 峰度偏度
        statistical_value_dict['峰度'].append(data[feature].kurt())
        statistical_value_dict['偏度'].append(data[feature].skew())

        # 箱线图
        iqr, up_limit, down_limit, outlier_num = box_outlier(data[feature])
        outlier_rate = outlier_num / sample_num
        statistical_value_dict['iqr(箱线图)'].append(iqr)
        statistical_value_dict['上限(箱线图)'].append(up_limit)
        statistical_value_dict['下限(箱线图)'].append(down_limit)
        statistical_value_dict['异常值数(箱线图)'].append(outlier_num)
        statistical_value_dict['异常值率(箱线图)'].append(outlier_rate)

        zero_num = data[data[feature] == 0].shape[0]
        zero_rate = zero_num / sample_num
        statistical_value_dict['零值数'].append(zero_num)
        statistical_value_dict['零值率'].append(zero_rate)

        positive_num = data[data[label] == 1].shape[0]
        positive_rate = positive_num / sample_num
```

```
        positive_missing_num = data[data[label] == 1][feature].isnull().sum()
        positive_missing_rate = positive_missing_num / positive_num
        positive_zero_num = data[(data[label] == 1) & (data[feature] == 0)][feature].shape[0]
        positive_zero_rate = positive_zero_num / positive_num
        statistical_value_dict['正样本数'].append(positive_num)
        statistical_value_dict['正样本率'].append(positive_rate)
        statistical_value_dict['正样本缺失数'].append(positive_missing_num)
        statistical_value_dict['正样本缺失率'].append(positive_missing_rate)
        statistical_value_dict['正样本零值数'].append(positive_zero_num)
        statistical_value_dict['正样本零值率'].append(positive_zero_rate)

        negative_num = data[data[label] == 0].shape[0]
        negative_rate = negative_num / sample_num
        negative_missing_num = data[data[label] == 0][feature].isnull().sum()
        negative_missing_rate = negative_missing_num / negative_num
        negative_zero_num = data[(data[label] == 0) & (data[feature] == 0)][feature].shape[0]
        negative_zero_rate = negative_zero_num / negative_num
        statistical_value_dict['负样本数'].append(negative_num)
        statistical_value_dict['负样本率'].append(negative_rate)
        statistical_value_dict['负样本缺失数'].append(negative_missing_num)
        statistical_value_dict['负样本缺失率'].append(negative_missing_rate)
        statistical_value_dict['负样本零值数'].append(negative_zero_num)
        statistical_value_dict['负样本零值率'].append(negative_zero_rate)

        var_value = np.var(data[feature])
        statistical_value_dict['方差'].append(var_value)

    statistical_value = pd.DataFrame(statistical_value_dict)

    result_data = pd.merge(result_data, statistical_value, on='字段名', how='left')
    return result_data


# a = univariate_analysis(data, list(data.columns)[:-1], 'y')
# a.to_excel('单变量分析.xlsx', index=None)

a = univariate_analysis(data, list(data.columns)[:-1], 'y')
```

```
a.to_excel('./save/单变量分析.xlsx', index=None)

```

1.4.2 输出样式

-

	字段名	计数_数据量	均值	标准差	最小值	第一四分位数(q1)	中位数	第三四分位数(q3)	最大值	极差	...	正样本缺失率	正样本零值数	正样本零值率	负样本数	负样本率
0	mean radius	569.0	14.127292	3.524049	6.98100	11.70000	13.37000	15.7800	28.1100	21.12900	...	0.0	0.0	0.0	212.0	0.372583
1	mean texture	569.0	19.289649	4.301036	9.71000	16.17000	18.84000	21.8000	39.2800	29.57000	...	0.0	0.0	0.0	212.0	0.372583
2	mean perimeter	569.0	91.969033	24.298981	43.79000	75.17000	86.24000	104.1000	188.5000	144.71000	...	0.0	0.0	0.0	212.0	0.372583
3	mean area	569.0	654.889104	351.914129	143.50000	420.30000	551.10000	782.7000	2501.0000	2357.50000	...	0.0	0.0	0.0	212.0	0.372583
4	mean smoothness	569.0	0.096360	0.014064	0.05263	0.08637	0.09587	0.1053	0.1634	0.11077	...	0.0	0.0	0.0	212.0	0.372583

1.5 其他小方法

1.5.1 随机打乱+恢复索引

- `sklearn.utils.shuffle`
- `作用`：随机打乱

```
from sklearn.datasets import load_iris
from sklearn.utils import shuffle
import pandas as pd

iris = load_iris()
data = pd.DataFrame(iris.data,columns=iris.feature_names)

# 随机打乱
data = shuffle(data,random_state=10)
data.head()
```

	sepal length (cm)	sepal width (cm)	petal length (cm)	petal width (cm)
点击滚动输出；双击隐藏		2.3	4.4	1.3
111	6.4	2.7	5.3	1.9
10	5.4	3.7	1.5	0.2
91	6.1	3.0	4.6	1.4
49	5.0	3.3	1.4	0.2

- 恢复索引

```
# 恢复索引
data.index = range(data.shape[0])
data.head()
```

	sepal length (cm)	sepal width (cm)	petal length (cm)	petal width (cm)
0	6.3	2.3	4.4	1.3
1	6.4	2.7	5.3	1.9
2	5.4	3.7	1.5	0.2
3	6.1	3.0	4.6	1.4
4	5.0	3.3	1.4	0.2

1.5.2 随机数种子

- 使用全局随机数种子，和使用random_state,当设置的随机数种子数相同时，两者所出的结果是相同的

1.5.2.1 `np.random.seed()`

- 作用：设置全局随机数种子

```
from sklearn.datasets import load_iris
from sklearn.utils import shuffle
import pandas as pd
import numpy as np

np.random.seed(10)

iris = load_iris()
data = pd.DataFrame(iris.data,columns=iris.feature_names)

# 随机打乱
data = shuffle(data
               #, random_state=10
               )
data.head()
```

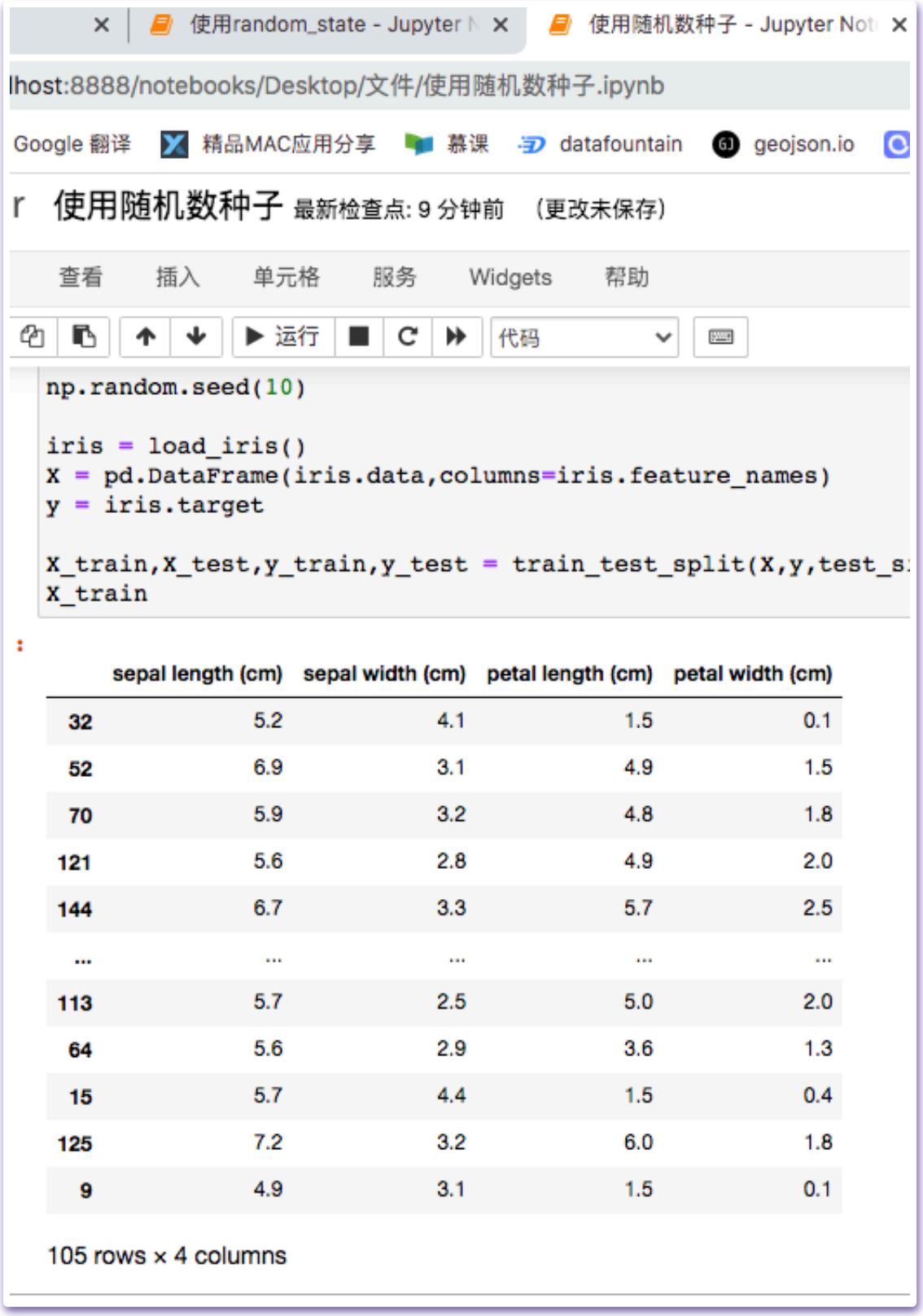
	sepal length (cm)	sepal width (cm)	petal length (cm)	petal width (cm)
87	6.3	2.3	4.4	1.3
111	6.4	2.7	5.3	1.9
10	5.4	3.7	1.5	0.2
91	6.1	3.0	4.6	1.4
49	5.0	3.3	1.4	0.2

结果与直接在 `shuffle` 函数中设置 `randon_state` 的效果是一样的

1.5.2.2 使用全局随机数种子

- 使用全局随机数种子，分训练集和测试集，与使用 `train_test_split` 中的 `random_state` 相对比

```
from sklearn.datasets import load_iris
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split

# 设置全局随机数种子
np.random.seed(10)

iris = load_iris()
X = pd.DataFrame(iris.data,columns=iris.feature_names)
y = iris.target

X_train,X_test,y_train,y_test = train_test_split(X
                                                 ,y
                                                 ,test_size=0.3
                                                )
X_train
```

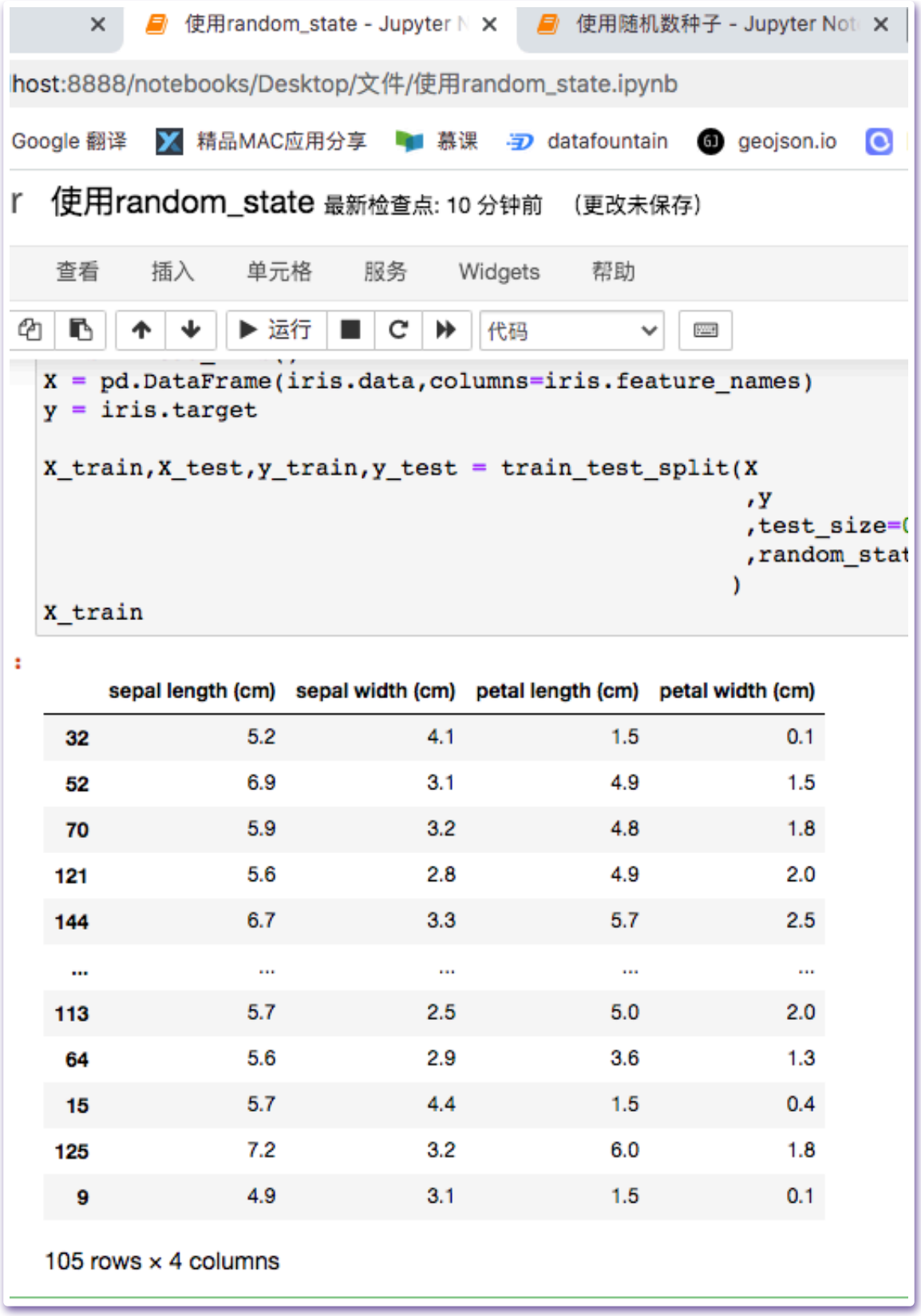


1.5.2.3 使用random_state

- 使用 `train_test_split` 中的 `random_state`，分训练集和测试集，与使用全局随机数种子相对比

```
from sklearn.datasets import load_iris
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
```

```python


iris = load_iris()
X = pd.DataFrame(iris.data,columns=iris.feature_names)
y = iris.target

X_train,X_test,y_train,y_test = train_test_split(X
                                                ,y
                                                ,test_size=0.3
                                                ,random_state=10
                                               )
X_train
```



	sepal length (cm)	sepal width (cm)	petal length (cm)	petal width (cm)
32	5.2	4.1	1.5	0.1
52	6.9	3.1	4.9	1.5
70	5.9	3.2	4.8	1.8
121	5.6	2.8	4.9	2.0
144	6.7	3.3	5.7	2.5
...	...	...	...	...
113	5.7	2.5	5.0	2.0
64	5.6	2.9	3.6	1.3
15	5.7	4.4	1.5	0.4
125	7.2	3.2	6.0	1.8
9	4.9	3.1	1.5	0.1

1.5.3 重命名列名

- `DataFrame.rename(columns={'列名A':'列名B'})`
- `作用`： 指定pandas的DataFrame中某一列变更列名

```python
from sklearn.datasets import load_iris
import pandas as pd

iris = load_iris()
data = pd.DataFrame(iris.data,columns=iris.feature_names)
data.head()
```

	sepal length (cm)	sepal width (cm)	petal length (cm)	petal width (cm)
0	5.1	3.5	1.4	0.2
1	4.9	3.0	1.4	0.2
2	4.7	3.2	1.3	0.2
3	4.6	3.1	1.5	0.2
4	5.0	3.6	1.4	0.2

```python
## 变更列名
data = data.rename(columns={'sepal length (cm)':'变更后的列名'})
data.head()
```

	变更后的列名	sepal width (cm)	petal length (cm)	petal width (cm)
0	5.1	3.5	1.4	0.2
1	4.9	3.0	1.4	0.2
2	4.7	3.2	1.3	0.2
3	4.6	3.1	1.5	0.2
4	5.0	3.6	1.4	0.2

2 二、连续变量分箱（二分类）

2.1 等距离分箱

2.1.1 等距离分箱_只输出分箱结果

2.1.1.1 代码

- 代码位置

> /Users/mengzhichao/Desktop/文件/16、python代码/Sample_Code/2、method/9、等距离分箱_只输出分箱结果.py

- 核心代码：

```
def equal_distances_bins(data, feature, n):
    cut_bins = pd.cut(data[feature], n)  # 等距离分箱
    cut_bins = pd.DataFrame(cut_bins.value_counts()).reset_index().sort_values('index').reset_index(drop=True)
    cut_bins.columns = ['分箱', '数据量']

    cut_bins['分箱_左'] = cut_bins['分箱'].astype('str').apply(lambda x: x.split(',')[0][1:])
    cut_bins['分箱_右'] = cut_bins['分箱'].astype('str').apply(lambda x: x.split(',')[1][:-1])

    cut_bins = cut_bins[['分箱', '分箱_左', '分箱_右', '数据量']]

    cut_bins['分箱_左'] = cut_bins['分箱_左'].astype('float')
    cut_bins['分箱_右'] = cut_bins['分箱_右'].astype('float')

    return cut_bins
```

- 所有代码：

```
# -*- coding: utf-8 -*-

"""
**************************************************
@author:   Ying
@software: PyCharm
@file: 9、等距离分箱_只输出分箱结果.py
@time: 2021-10-13 15:51
**************************************************
"""

import pandas as pd
```

```python
from sklearn.datasets import load_breast_cancer

wine = load_breast_cancer()
data = pd.DataFrame(wine.data, columns=wine.feature_names)
data['y'] = wine.target


def equal_distances_bins(data, feature, n):
    """
    等距离分箱

    :param data: 所有的数据
    :param feature: 需要分箱的特征
    :param n: 需要分多少箱
    :return: 返回分箱后的结果，其中分箱左、右界已经划分
    """
    cut_bins = pd.cut(data[feature], n)  # 等距离分箱
    cut_bins = pd.DataFrame(cut_bins.value_counts()).reset_index().sort_values('index').reset_index(drop=True)
    cut_bins.columns = ['分箱', '数据量']

    # 左右分箱的边界
    cut_bins['分箱_左'] = cut_bins['分箱'].astype('str').apply(lambda x: x.split(',')[0][1:])
    cut_bins['分箱_右'] = cut_bins['分箱'].astype('str').apply(lambda x: x.split(',')[1][:-1])

    cut_bins = cut_bins[['分箱', '分箱_左', '分箱_右', '数据量']]

    cut_bins['分箱_左'] = cut_bins['分箱_左'].astype('float')
    cut_bins['分箱_右'] = cut_bins['分箱_右'].astype('float')

    return cut_bins


a = equal_distances_bins(data, 'mean radius', 20)
print(a)
```

2.1.1.2 输出样式

-

2.1.2 等距离分箱_输出每条数据对应分箱

2.1.2.1 代码

- 代码位置：

> /Users/mengzhichao/Desktop/文件/16、python代码/Sample_Code/2、method/10、等

> 距离分箱_输出每条数据对应分箱.py

- 核心代码：

```
def equal_distances_bins_map(Y, X, bin_num):
    df1 = pd.DataFrame({"特征": X
                        , '标签': Y
                        , '所属分箱': pd.cut(X, bin_num)
                        }
                       )

    df1['分箱_左'] = df1['所属分箱'].astype('str').apply(lambda x: x.split(',')[0][1:])
    df1['分箱_右'] = df1['所属分箱'].astype('str').apply(lambda x: x.split(',')[1][:-1])

    return df1
```

- 所有代码：

```
# -*- coding: utf-8 -*-

"""
**************************************************
@author:   Ying
@software: PyCharm
@file: 10、等距离分箱_输出每条数据对应分箱.py
@time: 2021-10-13 16:26
**************************************************
"""
import pandas as pd
from sklearn.datasets import load_breast_cancer

wine = load_breast_cancer()
data = pd.DataFrame(wine.data, columns=wine.feature_names)
data['y'] = wine.target


def equal_distances_bins_map(Y, X, bin_num):
    """
    等距离分箱

    :param Y: 标签数据
    :param X: 需要分箱的特征数据
    :param bin_num: 需要分多少箱
    :return: 返回当前分箱特征每条数据对应分箱后的结果，其中分箱左、右界已经划分
    """

    # 等距离分箱
    df1 = pd.DataFrame({"特征": X
```

```
                                , '标签': Y
                                , '所属分箱': pd.cut(X, bin_num)
                                }
                               )

    # 左右分箱的边界
    df1['分箱_左'] = df1['所属分箱'].astype('str').apply(lambda x: x.split(',')[0][1:])
    df1['分箱_右'] = df1['所属分箱'].astype('str').apply(lambda x: x.split(',')[1][:-1])

    return df1


a = equal_distances_bins_map(data['y'], data['mean radius'], 20)
print(a)


```

2.1.2.2 输出样式

-

2.2 等频率分箱

2.2.1 等频率分箱_只输出分箱结果

2.2.1.1 代码

- 代码位置：

> /Users/mengzhichao/Desktop/文件/16、python代码/Sample_Code/2、method/11、等频率分箱_只输出分箱结果.py

- 核心代码

```
def equal_frequency_bins(data, feature, n):
    cut_bins = pd.qcut(data[feature], n)
    cut_bins = pd.DataFrame(cut_bins.value_counts()).reset_index().sort_values('index').reset_index(drop=True)
    cut_bins.columns = ['分箱', '数量']

    cut_bins['分箱_左'] = cut_bins['分箱'].astype('str').apply(lambda x: x.split(',')[0][1:])
    cut_bins['分箱_右'] = cut_bins['分箱'].astype('str').apply(lambda x: x.split(',')[1][:-1])

```

```
    cut_bins = cut_bins[['分箱', '分箱_左', '分箱_右', '数量']]

    cut_bins['分箱_左'] = cut_bins['分箱_左'].astype('float')
    cut_bins['分箱_右'] = cut_bins['分箱_右'].astype('float')

    return cut_bins

```

- 所有代码

```
import pandas as pd
from sklearn.datasets import load_breast_cancer

wine = load_breast_cancer()
data = pd.DataFrame(wine.data, columns=wine.feature_names)
data['y'] = wine.target


def equal_frequency_bins(data, feature, n):
    cut_bins = pd.qcut(data[feature], n)
    cut_bins = pd.DataFrame(cut_bins.value_counts()).reset_index().sort_values('index').reset_index(drop=True)
    cut_bins.columns = ['分箱', '数量']

    cut_bins['分箱_左'] = cut_bins['分箱'].astype('str').apply(lambda x: x.split(',')[0][1:])
    cut_bins['分箱_右'] = cut_bins['分箱'].astype('str').apply(lambda x: x.split(',')[1][:-1])

    cut_bins = cut_bins[['分箱', '分箱_左', '分箱_右', '数量']]

    cut_bins['分箱_左'] = cut_bins['分箱_左'].astype('float')
    cut_bins['分箱_右'] = cut_bins['分箱_右'].astype('float')

    return cut_bins


a = equal_frequency_bins(data, 'mean radius', 20)
print(a)

```

2.2.1.2 输出样式

2.2.2 等频率分箱_输出每条数据对应分箱

2.2.2.1 代码

- 代码位置：

> /Users/mengzhichao/Desktop/文件/16、python代码/Sample_Code/2、method/12、等频率分箱_输出每条数据对应分箱.py

- 核心代码

```
def equal_frequency_bins_map(Y, X, bin_num):

    df1 = pd.DataFrame({"特征": X
                        , '标签': Y
                        , '所属分箱': pd.qcut(X
                                              , bin_num)
                       }
                      )

    df1['分箱_左'] = df1['所属分箱'].astype('str').apply(lambda x: x.split(',')[0][1:])
    df1['分箱_右'] = df1['所属分箱'].astype('str').apply(lambda x: x.split(',')[1][:-1])

    return df1
```

- 所有代码

```
# -*- coding: utf-8 -*-

"""
**************************************************
@author:   Ying
@software: PyCharm
@file: 12、等频率分箱_输出每条数据对应分箱.py
@time: 2021-10-13 16:44
**************************************************
"""
import pandas as pd
from sklearn.datasets import load_breast_cancer

wine = load_breast_cancer()
data = pd.DataFrame(wine.data, columns=wine.feature_names)
data['y'] = wine.target


def equal_frequency_bins_map(Y, X, bin_num):
    # 等频率分箱
    df1 = pd.DataFrame({"特征": X
                        , '标签': Y
                        , '所属分箱': pd.qcut(X
                                              , bin_num)
                       }
                      )
```

```python

    df1['分箱_左'] = df1['所属分箱'].astype('str').apply(lambda x: x.split(',')[0][1:])
    df1['分箱_右'] = df1['所属分箱'].astype('str').apply(lambda x: x.split(',')[1][:-1])

    return df1


a = equal_frequency_bins_map(data['y'], data['mean radius'], 20)
print(a)

```

2.2.2.2 输出样式

3 三、模型相关

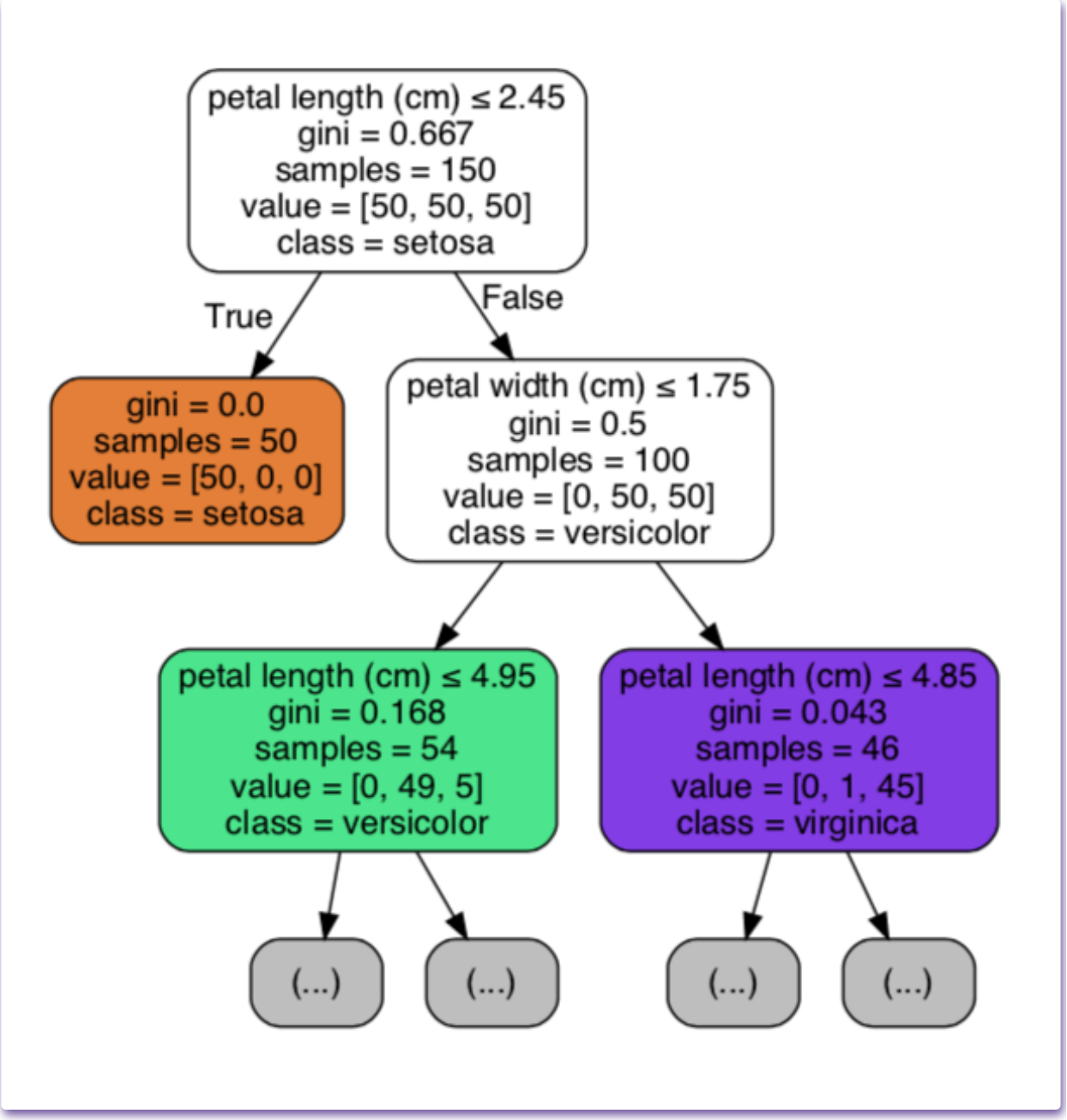
3.1 决策树可视化

3.1.1 graphviz 库

- 核心代码(这里只画出两层树用作演示)

```
dot_data = tree.export_graphviz(clf     # 决策树分类/回归器
                                ,feature_names=iris.feature_names   # 特征名称列表
                                ,class_names=iris.target_names   # 标签名称列表
                                ,rounded=True  # 圆角
                                ,filled=True  # 是否填充颜色
                                ,out_file=None
                                ,special_characters=True
                                ,max_depth=2 # 只显示两层树
                                )
graphviz.Source(dot_data)
```

- 输出样式



除叶节点（终端节点）之外的所有节点都有 5 部分

1. 基于一个特征的值的有关数据的问题。每个问题的答案要么是 True，要么就是 False。数据点会根据该问题的答案在该决策树中移动。
2. `gini`： 节点的基尼不纯度。当沿着树向下移动时，平均加权的基尼不纯度必须降低。
3. `samples`： 节点中观察的数量。
4. `value`： 每一类别中样本的数量。比如，顶部节点中有 2 个样本属于类别 0，有 4 个样本属于类别 1。
5. `class`： 节点中大多数点的类别（持平时默认为 0）。在叶节点中，这是该节点中所有样本的预测结果。

- 完整代码

```
from sklearn.datasets import load_iris
from sklearn import tree
```

```python
import graphviz
from sklearn.tree import DecisionTreeClassifier

iris = load_iris()

clf = DecisionTreeClassifier(criterion='gini')
clf.fit(iris.data,iris.target)

dot_data = tree.export_graphviz(clf    # 决策树分类/回归器
                            ,feature_names=iris.feature_names   # 特征名称列表
                            ,class_names=iris.target_names   # 标签名称列表
                            ,rounded=True  # 圆角
                            ,filled=True  # 是否填充颜色
                            ,out_file=None
                            ,special_characters=True
                            ,max_depth=2 # 只显示两层树
                            )


"""保存并实时查看"""
graph = graphviz.Source(dot_data)
graph.render(view=True # 是否实时展示
             ,format='pdf' # 保存的类型，pdf，png，jpg。。
             ,filename=f"decisiontree_pdf"  # 保存的文件名
             )

"""jupyter中查看"""

graphviz.Source(dot_data)
```

3.1.2 dtreeviz库

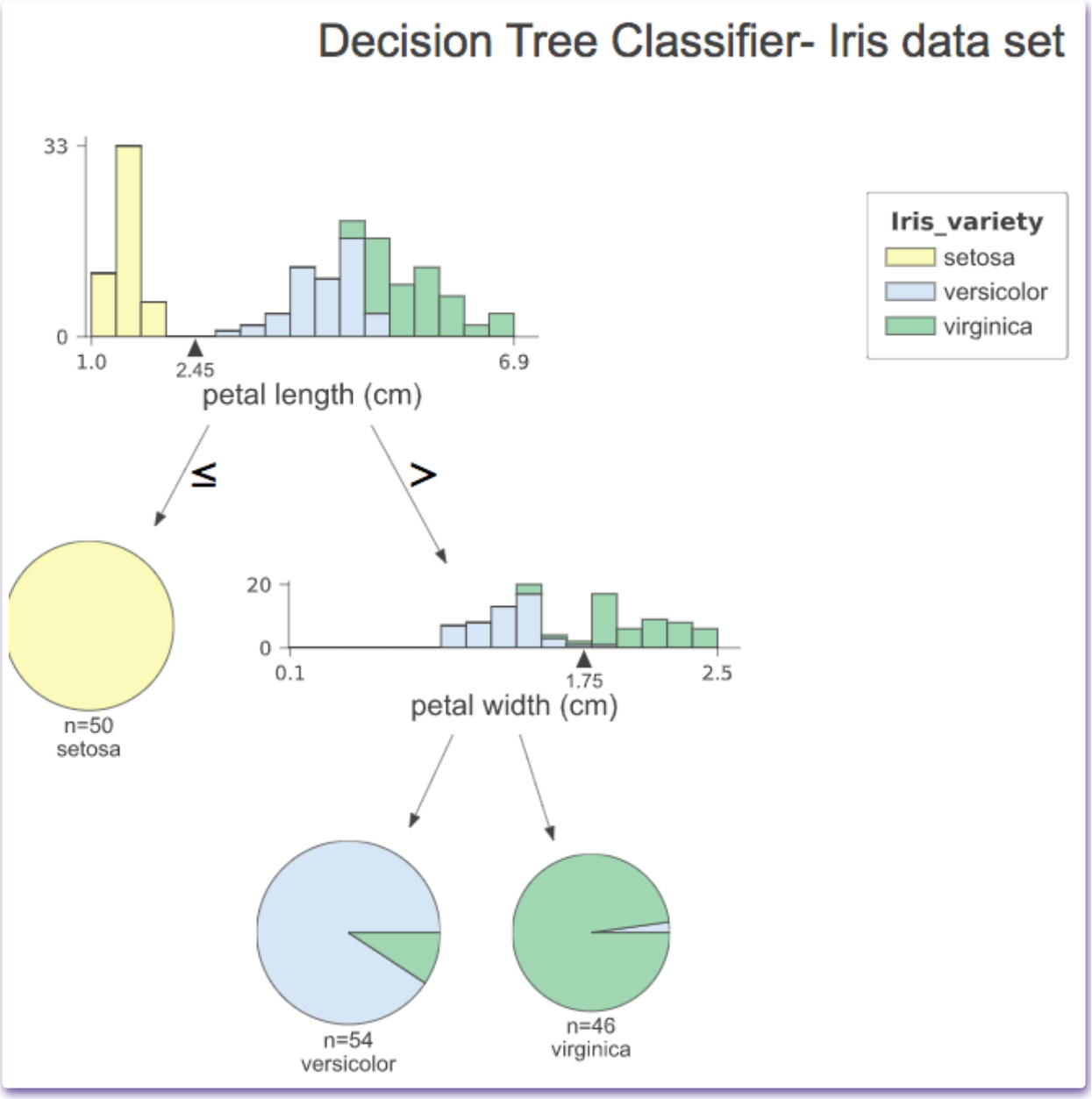
- `dtreeviz` 有以下特色：
    1. 利用有颜色的目标类别图例
    2. 叶子大小与该叶子中的样本数成正比
    3. 将≥和<用作边缘标签，看起来更清晰
    4. 决策节点利用堆叠直方图展示特征分布，每个目标类别都会用不同的颜色显示
    5. 在每个节点中各目标类别的样本数都用直方图的形式，这样可以提供更多信息

  `dtreeviz` 同样依赖GraphViz。
- `dtreeviz` 安装

```
pip install dtreeviz                    # install dtreeviz for sklearn

pip install dtreeviz[xgboost]           # install XGBoost related dependency

pip install dtreeviz[pyspark]           # install pyspark related dependency

pip install dtreeviz[lightgbm]          # install LightGBM related dependency
```

- 分类树

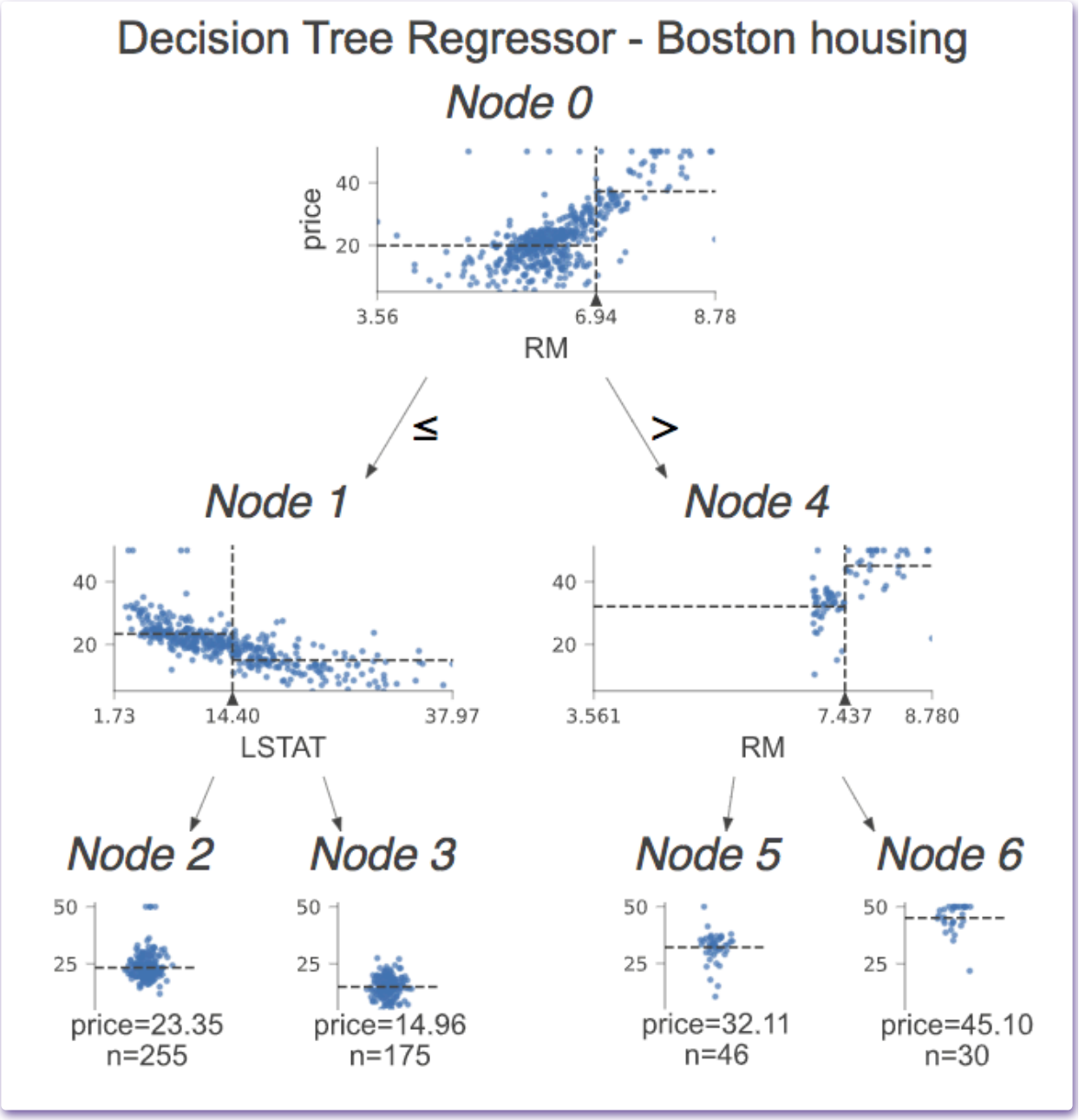
```
"""分类树"""

from sklearn.datasets import load_iris
from sklearn import tree
import graphviz
from dtreeviz.trees import dtreeviz
from sklearn.tree import DecisionTreeClassifier

iris = load_iris()

clf = DecisionTreeClassifier(criterion='gini',max_depth=2)# 演示用，只用2层树
clf.fit(iris.data,iris.target)

viz = dtreeviz(clf              #分类器
               ,iris.data       # X
               ,iris.target     # y
               ,target_name='Iris_variety'
               ,feature_names=iris.feature_names    # 特征名称
               ,class_names={0:'setosa',1:'versicolor',2:'virginica'}    # 类别名称
               ,title="Decision Tree Classifier- Iris data set"    # 标题名称
               ,scale=2.4   # 缩放比例，数值越大图越大
               )


viz
```



- 回归树

```
"""回归树"""
from sklearn.datasets import load_boston
from sklearn import tree
import graphviz
from dtreeviz.trees import dtreeviz
from sklearn.tree import DecisionTreeRegressor

boston = load_boston()

reg = DecisionTreeRegressor(criterion='mse',max_depth=2)# 演示用，只用2层树
reg.fit(boston.data,boston.target)



viz = dtreeviz(reg    # 回归器
               ,x_data=boston.data
```

```
                    ,y_data=boston.target
                    ,target_name='price'
                    ,feature_names=boston.feature_names
                    ,title="Decision Tree Regressor - Boston housing"
                    ,show_node_labels = True
                    ,scale=2.4
                   )



viz
```



3.2 随机森林填补缺失值（回归）

- 首先，现实中基本上不会用随机森林填补缺失值，主要原因有下

1. 现实数据可能是海量，随机森林会算到天荒地老
2. 随机森林算法不容易被你的老板，朋友理解

所以最常见的填充方法就是：

1. `连续型用均值填充` (训练集和测试集的均值可能不同，用训练集的均值同时对训练集和测试集进行填充)
2. `分类型用众数填充` (训练集和测试集的众数可能不同，用训练集的众数同时对训练集和测试集进行填充)

- 核心代码

```
def fill_missing_rf(X,y,to_fill):
    """
    使用 随机森林回归 填补一个特征的缺失值函数

    参数:
    X:要填补的特征矩阵
    y:完整的，没有缺失值的标签
    to_fill:字符串，要填补的那一列名称
    """

    # 构建新特征矩阵和新标签
    df = X.copy()
    fill = df.loc[:,to_fill]
    df = pd.concat([df.loc[:,df.columns != to_fill],pd.DataFrame(y)],axis=1)

    # 找出训练集和测试集
    y_train = fill[fill.notnull()]
    y_test = fill[fill.isnull()]
    x_train = df.iloc[y_train.index,:]
    x_test = df.iloc[y_test.index,:]

    # 用随机森林回归来填补缺失值
    rfr = RFR(n_estimators=100)
    rfr = rfr.fit(x_train,y_train)
    y_predict = rfr.predict(x_test)

    return y_predict
```

- 所有代码

```
# -*- coding: utf-8 -*-

"""
**************************************************
@author:   Ying
@software: PyCharm
@file: 16、随机森林回归填补缺失值.py
@time: 2022-02-21 15:23
**************************************************
```

```python
"""

import numpy as np
from sklearn.datasets import load_boston
from sklearn.ensemble import RandomForestRegressor as RFR
import pandas as pd

# 用波士顿房价数据集来演示，将AGE字段的五分之一设置为缺失值
boston = load_boston()
data = pd.DataFrame(boston.data, columns=boston.feature_names)
data['y'] = boston.target

isnull_num = np.arange(0, data.shape[0], 5)
data.loc[isnull_num, 'AGE'] = np.nan


def fill_missing_rfr(X, y, to_fill):
    """
    使用 随机森林回归 填补一个特征的缺失值函数

    参数:
    X:要填补的特征矩阵
    y:完整的，没有缺失值的标签
    to_fill:字符串，要填补的那一列名称
    """

    # 构建新特征矩阵和新标签
    df = X.copy()
    fill = df.loc[:, to_fill]
    df = pd.concat([df.loc[:, df.columns != to_fill], pd.DataFrame(y)], axis=1)

    # 找出训练集和测试集
    y_train = fill[fill.notnull()]
    y_test = fill[fill.isnull()]
    x_train = df.iloc[y_train.index, :]
    x_test = df.iloc[y_test.index, :]

    # 用随机森林回归来填补缺失值
    rfr = RFR(n_estimators=100)
    rfr = rfr.fit(x_train, y_train)
    y_predict = rfr.predict(x_test)

    return y_predict


X = data.iloc[:, :-1]
y = data.iloc[:, -1]
# X.isnull().mean()

age_pred = fill_missing_rfr(X, y, 'AGE')
data.loc[data.loc[:, 'AGE'].isnull(), 'AGE'] = age_pred
```

```

# data.isnull().mean()

```

3.3 模型保存

3.3.1 joblib库

- 函数作用： 将模型保存
- dump 、 load

```python
# -*- coding: utf-8 -*-

"""
**************************************************
@author:   Ying
@software: PyCharm
@file: 17、模型保存_joblib.py
@time: 2022-02-25 16:56
**************************************************
"""

from sklearn.ensemble import RandomForestClassifier
from joblib import dump, load

rfc = RandomForestClassifier(random_state=0
                             , n_estimators=10)

# dump(model,'model_name.model')  # 存储模型
dump(rfc, './save/rfc.dat')

# load('model_name.model')       # 加载模型
load_model = load('./save/rfc.dat')
print(load_model)




"""
RandomForestClassifier(n_estimators=10, random_state=0)
"""
```

3.3.2 pickle

- 函数作用： 保存模型

```python
# -*- coding: utf-8 -*-
```

```python

"""
**************************************************
@author:   Ying
@software: PyCharm
@file: 18、模型保存_pickle.py.py
@time: 2022-02-25 17:28
**************************************************
"""

from sklearn.ensemble import RandomForestClassifier
import pickle

rfc = RandomForestClassifier(random_state=0
                             , n_estimators=10)

# 保存模型
pickle.dump(rfc, open("rfc_pickle.dat", "wb"))

# 导入模型
load_model = pickle.load(open("rfc_pickle.dat", "rb"))

print(load_model)


"""
RandomForestClassifier(n_estimators=10, random_state=0)
"""
```

4 四、日期相关

4.1 FuncDateReFormat

- `函数作用`：修改日期的格式
- `FuncDateReFormat`

```python
def FuncDateReFormat(datestr, fromFormat, toFormat):
    """
    函数作用：修改日期的格式，如yyyymmdd -->  yyyy-mm-dd
    :param datestr: 原始日期【字符串】
    :param formFormat: 原始格式，如yyyymmdd
    :param toFormat: 目标格式，如yyyy-mm-dd、yyyy-mm-01
    :return: 目标格式日期【字符串】
    """
    date_str = datetime.datetime.strptime(datestr, fromFormat)
    return date_str.strftime(toFormat)
```

- `示例`

```python
FuncDateReFormat('2021-03-21','%Y-%m-%d','%Y-%m-01')

"""输出"""
'2021-03-01'
```

```python
FuncDateReFormat('20210321','%Y%m%d','%Y-%m-01')

"""输出"""
'2021-03-01'
```

```python
FuncDateReFormat('20210321','%Y%m%d','%Y-%m-%d')

"""输出"""
'2021-03-21'
```

```python
FuncDateReFormat('2021-03-21','%Y-%m-%d','%Y%m%d')

"""输出"""
'20210321'
```

4.2 FuncLastDay

- 函数`作用`：返回当月最后一天
- `FuncLastDay`

```python
def FuncLastDay(datestr, dateFormat='%Y-%m-%d'):
    """
    函数作用：返回日期所在月份的最后一天（字符串）
    :param datestr: 原始日期【字符串】
    :param dateFormat: 原始格式
    :return: 日期所在当月的最后一天
    """
    date_str = FuncDateReFormat(datestr, dateFormat, "%Y-%m-01")  # 返回当月日期第一天【字符串】
    date_str = datetime.datetime.strptime(datestr, "%Y-%m-%d")  # 转换为日期
    date_str += pd.DateOffset(months=1)  # 返回下月第一天
    date_str += pd.DateOffset(days=-1)  # 返回当月最优一天【下个月第一天的前一天】
    return date_str.strftime(dateFormat)
```

- 示例

```python
FuncLastDay('2021-03-21')

"""输出"""
'2021-03-31'
```

```python
FuncLastDay('20210321','%Y%m%d')

"""输出"""
'20210331'
```

4.3 FuncDateAdd

- 函数作用：日期计算，加减若干天，加减若干月
- FuncDateAdd

```python
def FuncDateAdd(datestr, add_num, date_intvl='DD', date_format='%Y-%m-%d'):
    """
    函数作用：日期计算，加减若干天，加减若干月
    :param datestr: 原始字符串
    :param add_num: 待增加的天数、月数
    :param date_intvl: 增加频率 天、月 【待选项DD、MM、MMT】
    :param date_format: 日期格式
    :return:
        1. 若 date_intvl='DD'，返回增加、相减了相应天数的日期
        2. 若 date_intvl='MM'，返回增加、相减了相应月数的日期
        3. 若 date_intvl='MMT'，返回增加、相减了相应月数的日期当月的第一天

    """
    date_str = datetime.datetime.strptime(datestr, date_format)
    if date_intvl.upper() == "DD":  # 返回增加、相减了相应天数的日期
        date_str += pd.DateOffset(days=add_num)
```

```
        return date_str.strftime(date_format)
    elif date_intvl.upper() == "MM":  # 返回增加、相减了相应月数数的日期
        date_str += pd.DateOffset(months=add_num)
        date_str_res = date_str.strftime(date_format)
        if FuncLastDay(datestr, date_format) == date_str:
            date_str_res = FuncLastDay(date_str_res)
        return date_str_res

    elif date_intvl.upper() == 'MMT':  # 返回增加、相减了相应月数的日期当月的第一天
        date_str += pd.DateOffset(months=add_num)
        datestr = date_str.strftime('%Y-%m-01')
        date_str = FuncDateReFormat(date_str, "%Y-%m-%d", date_format)
        return date_str
    else:
        return None
```

- 示例

  1. 若 `date_intvl='DD'`，返回增加、相减了相应天数的日期

     ```
     FuncDateAdd(datestr='2021-03-21',add_num=5,date_intvl='DD')

     """输出"""
     '2021-03-26'
     ```

  2. 若 `date_intvl='MM'`，返回增加、相减了相应月数的日期

     ```
     FuncDateAdd(datestr='2021-03-21',add_num=5,date_intvl='MM')

     """输出"""
     '2021-08-21'
     ```

  3. 若 `date_intvl='MMT'`，返回增加、相减了相应月数的日期当月的第一天

     ```
     FuncDateAdd(datestr='2021-03-21',add_num=5,date_intvl='MMT')

     """输出"""
     '2021-08-01'
     ```

4.4 FuncGenDateList

- 函数作用：传入日期间隔【天、月】返回开始日期与结束日期之间的所有日期字符串列表
- 备注：包头不包尾
- `FuncGenDateList`

```python
def FuncGenDateList(start_date_str, end_date_str, date_intvl="DD", add_num=1, date_format='%Y-%m-%d'):
    """
    函数作用：传入日期间隔【天、月】返回开始日期与结束日期之间的所有日期字符串列表
    :param start_date_str: 开始日期【字符串】
    :param end_date_str: 结束日期【字符串】
    :param date_intvl: 增加频率 天、月【待选项DD、MM、MMT】
    :param add_num: 待增加的天数、月数
    :param date_format: 日期格式
    :return:
    """
    res = []
    date_intvl_cals = date_intvl[:2]  # 字符串截取，如果是MMT则去MM，MMT在下面处理
    if start_date_str < end_date_str:
        while (start_date_str < end_date_str):
            res.append(start_date_str)
            start_date_str = FuncDateAdd(start_date_str, add_num, date_intvl_cals, date_format)

    if date_intvl == "MMT":  # 处理MMT
        res = [FuncDateAdd(x, 1, 'MM', date_format) for x in res]  # 加一个月
        res = [FuncDateReFormat(x, date_format, "%Y-%m-01") for x in res]  # 当月第一天
        res = [FuncDateAdd(x, -1, "DD") for x in res]  # 前一个月最后一天
        res = [FuncDateReFormat(x, '%Y-%m-%d', date_format) for x in res]

    return res
```

- 示例：

  1. 如果开始日期 >= 结束日期，返回的是空列表

```python
FuncGenDateList(start_date_str='2021-03-21',end_date_str='2021-03-20',date_intvl='DD')

"""输出"""
[]
```

```python
FuncGenDateList(start_date_str='2021-03-21',end_date_str='2021-03-21',date_intvl='DD')

"""输出"""
[]
```

  2. 如果开始日期>结束日期，`date_intvl = “DD”`，返回从开始日期到结束日期之间所有日期的列表（日维度）

```
FuncGenDateList(start_date_str='2021-03-21',end_date_str='2021-03-25',date_intvl='DD')

"""输出"""
['2021-03-21', '2021-03-22', '2021-03-23', '2021-03-24']
```

3. 如果开始日期>结束日期，`date_intvl = “MM”`，返回从开始日期到结束日期之间所有日期的列表（月维度）

```
FuncGenDateList(start_date_str='2021-03-21',end_date_str='2021-03-31',date_intvl='MM')

"""输出"""
['2021-03-21']
```

```
FuncGenDateList(start_date_str='2021-03-21',end_date_str='2021-05-31',date_intvl='MM')

"""输出"""
['2021-03-21', '2021-04-21', '2021-05-21']
```

4. 如果开始日期>结束日期，`date_intvl = “MMT”`，返回从开始日期到结束日期之间所有日期单月最后一天的列表（月维度）

```
FuncGenDateList(start_date_str='2021-03-21',end_date_str='2021-03-25',date_intvl='MMT')

"""输出"""
['2021-03-31']
```

```
FuncGenDateList(start_date_str='2021-03-21',end_date_str='2021-05-25',date_intvl='MMT')

"""输出"""
['2021-03-31', '2021-04-30', '2021-05-31']
```

4.5 对系统当前时间操作

- `current_date_str` 但会当前系统日期字符串

```
current_date_str = datetime.datetime.now().strftime('%Y-%m-%d')
current_date_str

"""输出"""
'2021-12-31'
```

- `current_time_str` 返回当前系统时间字符串

```
current_time_str = time.strftime('%T')
current_time_str

"""输出"""
'16:42:34'
```

- `current_month_str` 返回当月第一天日期字符串

```
current_month_str = datetime.datetime.now().strftime('%Y-%m-01')
current_month_str

"""输出"""
'2021-12-01'
```

- `last_date_str` 返回前一天日期（调用了上面的 `FuncDateAdd` 函数）

```
last_date_str = FuncDateAdd(current_date_str, -1, 'DD')
last_date_str

"""输出"""
'2021-12-30'
```

- `last_month_str` 返回 `current_month_str` 的上一个月（调用它了上面的 `FuncDateAdd` 函数）

```
last_month_str = FuncDateAdd(current_date_str, -1, 'MM')
last_month_str

"""输出"""
'2021-11-30'
```

4.6 综合

- 放在同一个py文件中

```
# -*- coding: utf-8 -*-

"""
**************************************************
@author:   Ying
@software: PyCharm
@file: 15、日期相关处理.py
@time: 2021-12-31 15:07
**************************************************
"""

```

```python
import datetime
import time
import pandas as pd


def FuncDateReFormat(datestr, fromFormat, toFormat):
    """
    函数作用：修改日期的格式，如yyyymmdd -->  yyyy-mm-dd
    :param datestr: 原始日期【字符串】
    :param formFormat: 原始格式，如yyyymmdd
    :param toFormat: 目标格式，如yyyy-mm-dd、yyyy-mm-01
    :return: 目标格式日期【字符串】
    """
    date_str = datetime.datetime.strptime(datestr, fromFormat)
    return date_str.strftime(toFormat)


def FuncLastDay(datestr, dateFormat='%Y-%m-%d'):
    """
    函数作用：返回日期所在月份的最后一天（字符串）
    :param datestr: 原始日期【字符串】
    :param dateFormat: 原始格式
    :return: 日期所在当月的最后一天
    """
    date_str = FuncDateReFormat(datestr, dateFormat, "%Y-%m-01")  # 返回当月日期第一天【字符串】
    date_str = datetime.datetime.strptime(date_str, "%Y-%m-%d")  # 转换为日期
    date_str += pd.DateOffset(months=1)  # 返回下月第一天
    date_str += pd.DateOffset(days=-1)  # 返回当月最优一天【下个月第一天的前一天】
    return date_str.strftime(dateFormat)


def FuncDateAdd(datestr, add_num, date_intvl='DD', date_format='%Y-%m-%d'):
    """
    函数作用：日期计算，加减若干天，加减若干月
    :param datestr: 原始字符串
    :param add_num: 待增加的天数、月数
    :param date_intvl: 增加频率 天、月 【待选项DD、MM、MMT】
    :param date_format: 日期格式
    :return:
        1. 若 date_intvl='DD'，返回增加、相减了相应天数的日期
        2. 若 date_intvl='MM'，返回增加、相减了相应月数的日期
        3. 若 date_intvl='MMT'，返回增加、相减了相应月数的日期当月的第一天

    """
    date_str = datetime.datetime.strptime(datestr, date_format)
    if date_intvl.upper() == "DD":  # 返回增加、相减了相应天数的日期
        date_str += pd.DateOffset(days=add_num)
```

```python
        return date_str.strftime(date_format)
    elif date_intvl.upper() == "MM":  # 返回增加、相减了相应月数数的日期
        date_str += pd.DateOffset(months=add_num)
        date_str_res = date_str.strftime(date_format)
        if FuncLastDay(datestr, date_format) == date_str:
            date_str_res = FuncLastDay(date_str_res)
        return date_str_res

    elif date_intvl.upper() == 'MMT':  # 返回增加、相减了相应月数的日期当月的第一天
        date_str += pd.DateOffset(months=add_num)
        date_str = date_str.strftime('%Y-%m-01')
        date_str = FuncDateReFormat(date_str, "%Y-%m-%d", date_format)
        return date_str
    else:
        return None


def FuncGenDateList(start_date_str, end_date_str, date_intvl="DD", add_num=1, date_format='%Y-%m-%d'):
    """
    函数作用：传入日期间隔【天、月】返回开始日期与结束日期之间的所有日期字符串列表
    :param start_date_str: 开始日期【字符串】
    :param end_date_str: 结束日期【字符串】
    :param date_intvl: 增加频率 天、月【待选项DD、MM、MMT】
    :param add_num: 待增加的天数、月数
    :param date_format: 日期格式
    :return:
    """
    res = []
    date_intvl_cals = date_intvl[:2]  # 字符串截取，如果是MMT则去MM，MMT在下面处理
    if start_date_str < end_date_str:
        while (start_date_str < end_date_str):
            res.append(start_date_str)
            start_date_str = FuncDateAdd(start_date_str, add_num, date_intvl_cals, date_format)

    if date_intvl == "MMT":  # 处理MMT
        res = [FuncDateAdd(x, 1, 'MM', date_format) for x in res]  # 加一个月
        res = [FuncDateReFormat(x, date_format, "%Y-%m-01") for x in res]  # 当月第一天
        res = [FuncDateAdd(x, -1, "DD") for x in res]  # 前一个月最后一天
        res = [FuncDateReFormat(x, '%Y-%m-%d', date_format) for x in res]

    return res
```

```
100
101
102 current_date_str = datetime.datetime.now().strftime('%Y-%m-%d')
103
104 current_time_str = time.strftime('%T')
105
106 current_month_str = datetime.datetime.now().strftime('%Y-%m-01')
107
108 last_date_str = FuncDateAdd(current_date_str, -1, 'DD')
109
110 last_month_str = FuncDateAdd(current_date_str, -1, 'MM')
```

5 五、风控、机器学习相关

- 建模过程中，经常采用预测性（iv）、相关性（pearson）、共线性（vif）等维度来选择合适的特征变量
- 树模型经常使用自己的feature_importances来选择

5.1 woe分箱_iv值计算

- 基于 `scorecardpy` 库，乳腺癌数据集

```
import pandas as pd
import numpy as np
from sklearn.datasets import load_breast_cancer
import scorecardpy as sc
from tqdm import notebook

cancer = load_breast_cancer()
df = pd.DataFrame(cancer.data,columns=['_'.join(i.split()) for i in cancer.feature_names])
df['y'] = cancer.target

"""woe计算"""
#所有的column是
need_columns = df.iloc[:,:-1].columns.tolist()
data_list = []
for i in notebook.tqdm(need_columns):
    bins = sc.woebin(df[[i,'y']],y='y')
    data_list.append(bins[i])

result_woe = pd.concat(data_list)

result_woe.columns = ['特征','分箱','分箱样本数','count_distr','负样本数'
                      ,'正样本数','正样本占比_当前分箱','woe_当前分箱'
                      ,'iv_当前分箱','iv值','分割点','是否特殊值']

result_woe['负样本占比_当前分箱'] = result_woe['负样本数']/result_woe['分箱样本数']
result_woe['正样本%'] = result_woe['正样本数']/df['y'].sum()
result_woe['负样本%'] = result_woe['负样本数']/(df.shape[0] - df['y'].sum())

result_woe = result_woe[['特征', '分箱', '分割点', '是否特殊值', '分箱样本数'
                      , 'count_distr', '负样本数', '正样本数', '正样本占比_当前分箱'
                      , '负样本占比_当前分箱', '正样本%','负样本%','woe_当前分箱', 'iv_当前分箱', 'iv值']]
```

特征	分箱	分割点	是否特殊值	分箱样本数	count_distr	负样本数	正样本数	正样本占比_当前分箱	负样本占比_当前分箱	正样本%	负样本%	woe_当前分箱	iv_当前分箱	iv值
mean_radius	[-inf,13.0)	13.0	FALSE	256	0.449912127	12	244	95.31%	4.69%	68.35%	5.66%	2.4911	1.5616	4.8197
mean_radius	[13.0,15.0)	15.0	FALSE	139	0.244288225	39	100	71.94%	28.06%	28.01%	18.40%	0.4205	0.0404	4.8197
mean_radius	[15.0,17.0)	17.0	FALSE	56	0.098418278	44	12	21.43%	78.57%	3.36%	20.75%	-1.8204	0.3166	4.8197
mean_radius	[17.0,inf)	inf	FALSE	118	0.207381371	117	1	0.85%	99.15%	0.28%	55.19%	-5.2833	2.9010	4.8197
mean_texture	[-inf,15.0)	15.0	FALSE	91	0.159929701	4	87	95.60%	4.40%	24.37%	1.89%	2.5585	0.5752	1.2107
mean_texture	[15.0,19.0)	19.0	FALSE	203	0.356766257	45	158	77.83%	22.17%	44.26%	21.23%	0.7348	0.1692	1.2107
mean_texture	[19.0,20.0)	20.0	FALSE	50	0.087873462	21	29	58.00%	42.00%	8.12%	9.91%	-0.1984	0.0035	1.2107
mean_texture	[20.0,inf)	inf	FALSE	225	0.39543058	142	83	36.89%	63.11%	23.25%	66.98%	-1.0581	0.4627	1.2107
mean_perimeter	[-inf,72.0)	72.0	FALSE	104	0.182776801	1	103	99.04%	0.96%	28.85%	0.47%	4.1136	1.1674	5.1085
mean_perimeter	[72.0,90.0)	90.0	FALSE	227	0.398945518	22	205	90.31%	9.69%	57.42%	10.38%	1.7108	0.8049	5.1085
mean_perimeter	[90.0,110.0	110.0	FALSE	118	0.207381371	70	48	40.68%	59.32%	13.45%	33.02%	-0.8984	0.1759	5.1085
mean_perimeter	[110.0,inf)	inf	FALSE	120	0.210896309	119	1	0.83%	99.17%	0.28%	56.13%	-5.3003	2.9603	5.1085
mean_area	[-inf,400.0	400.0	FALSE	118	0.207381371	3	115	97.46%	2.54%	32.21%	1.42%	3.1252	0.9625	4.7152
mean_area	[400.0,500.	500.0	FALSE	112	0.196836555	7	105	93.75%	6.25%	29.41%	3.30%	2.1869	0.5710	4.7152
mean_area	[500.0,700	700.0	FALSE	168	0.295254833	41	127	75.60%	24.40%	35.57%	19.34%	0.6095	0.0989	4.7152
mean_area	[700.0,950.	950.0	FALSE	67	0.117750439	58	9	13.43%	86.57%	2.52%	27.36%	-2.3844	0.5922	4.7152
mean_area	[950.0,inf)	inf	FALSE	104	0.182776801	103	1	0.96%	99.04%	0.28%	48.58%	-5.1559	2.4905	4.7152
mean_smoothness	[-inf,0.080	0.0800000	FALSE	62	0.108963093	3	59	95.16%	4.84%	16.53%	1.42%	2.4578	0.3714	0.8196
mean_smoothness	[0.0800000	0.0900000	FALSE	137	0.240773286	26	111	81.02%	18.98%	31.09%	12.26%	0.9303	0.1752	0.8196
mean_smoothness	[0.0900000	0.1	FALSE	153	0.268892794	62	91	59.48%	40.52%	25.49%	29.25%	-0.1374	0.0052	0.8196
mean_smoothness	[0.1,0.1100	0.1100000	FALSE	131	0.230228471	65	66	50.38%	49.62%	18.49%	30.66%	-0.5059	0.0616	0.8196
mean_smoothness	[0.1100000	inf	FALSE	86	0.151142355	56	30	34.88%	65.12%	8.40%	26.42%	-1.1453	0.2063	0.8196
mean_compactness	[-inf,0.05)	0.05	FALSE	60	0.105448155	1	59	98.33%	1.67%	16.53%	0.47%	3.5564	0.5710	2.2118
mean_compactness	[0.05,0.08)	0.08	FALSE	169	0.297012302	18	151	89.35%	10.65%	42.30%	8.49%	1.6058	0.5428	2.2118
mean_compactness	[0.08,0.099	0.0999999	FALSE	78	0.137082601	17	61	78.21%	21.79%	17.09%	8.02%	0.7565	0.0686	2.2118

- `woe` 计算公式

$$woe = \log(\frac{good\%}{bad\%}) \tag{1}$$

- `iv` 计算公式

$$iv = \sum(good\% - bad\%) * woe \tag{2}$$

5.2 vif-方差膨胀因子

- 使用 `statsmodels` 中的 `variance_inflation_factor`，数据集使用乳腺癌数据集

```python
import pandas as pd
import numpy as np
from sklearn.datasets import load_breast_cancer
from tqdm import notebook
from statsmodels.stats.outliers_influence import variance_inflation_factor
from statsmodels.tools import add_constant

cancer = load_breast_cancer()
df = pd.DataFrame(cancer.data,columns=['_'.join(i.split()) for i in cancer.feature_names])
df['y'] = cancer.target

def calc_vif(df,desc_dict=None):
    """
    df: DataFrame
    desc_dict: 字段中文释义字典

    return: vif DataFrame
    """
```

```
    X=add_constant(df)
    vif_list = []
    for i in notebook.tqdm(range(X.shape[1])):
        vif = variance_inflation_factor(X.values,i)
        vif_list.append(vif)
    s = pd.DataFrame(vif_list,index=X.columns)
    s = s.reset_index()
    s.columns = ['feature','vif']
    if desc_dict:
        s['中文释义'] = s.feature.map(desc_dict)
        s = s[['feature','中文释义','vif']]
    return s

str_list = df.select_dtypes(include='object').columns.tolist() # string类型
flo_list = df.select_dtypes(exclude='object').columns.tolist() # 连续型

# vif 不能计算string类型
df_vif = df[flo_list]

# vif 计算不能有空值
for i in notebook.tqdm(flo_list):
    df_vif[i] = df_vif[i].fillna(0)

vif_data = calc_vif(df_vif)
vif_data
```

```
vif_data
```

executed in 22ms, finished 16:15:26 2023-01-04

	feature	vif
0	const	2041.278511
1	mean_radius	3817.259795
2	mean_texture	11.891280
3	mean_perimeter	3792.697001
4	mean_area	348.115385
5	mean_smoothness	8.194309
6	mean_compactness	51.445960
7	mean_concavity	71.002747
8	mean_concave_points	60.172431
9	mean_symmetry	4.220806
10	mean_fractal_dimension	15.756978
11	radius_error	75.737325
12	texture_error	4.205686
13	perimeter_error	70.398925
14	area_error	41.196467
15	smoothness_error	4.070801
16	compactness_error	15.366350
17	concavity_error	15.914022
18	concave_points_error	11.601253
19	symmetry_error	5.179151
20	fractal_dimension_error	9.724753

5.3 KS值和KS曲线

- KS在风控主要是用于评估模型的好坏样本区分度高低的,KS的值域在0-1之间，一般来说KS是越大越有区分度的，但在风控领域并不是越大越好:

KS取值范围	风控模型可用性说明
<0.2	模型不可用
0.2-0.3	模型有一定效果，一般见于运行很久的模型
0.3-0.5	模型可用
0.5-0.6	模型很强！
0.6-0.7	模型存疑！不建议用
>0.7	模型基本不可用，肯定有什么地方错了

- `ks` 值

```
import pandas as pd
import numpy as np
from matplotlib import pyplot as plt

from scipy.stats import ks_2samp
from sklearn import metrics

y_true = np.array([1,1,0,0,0,0,0,0,0,0])
y_pred_prob = np.array([0.7,0.6,0.1,0.3,0.3,0.5,0.6,0.4,0.1,0.2])

"""1、用scipy库计算"""
ks1 = ks_2samp(y_pred_prob[y_true==1],y_pred_prob[y_true==0]).statistic
ks1

"""输出"""
0.875
```

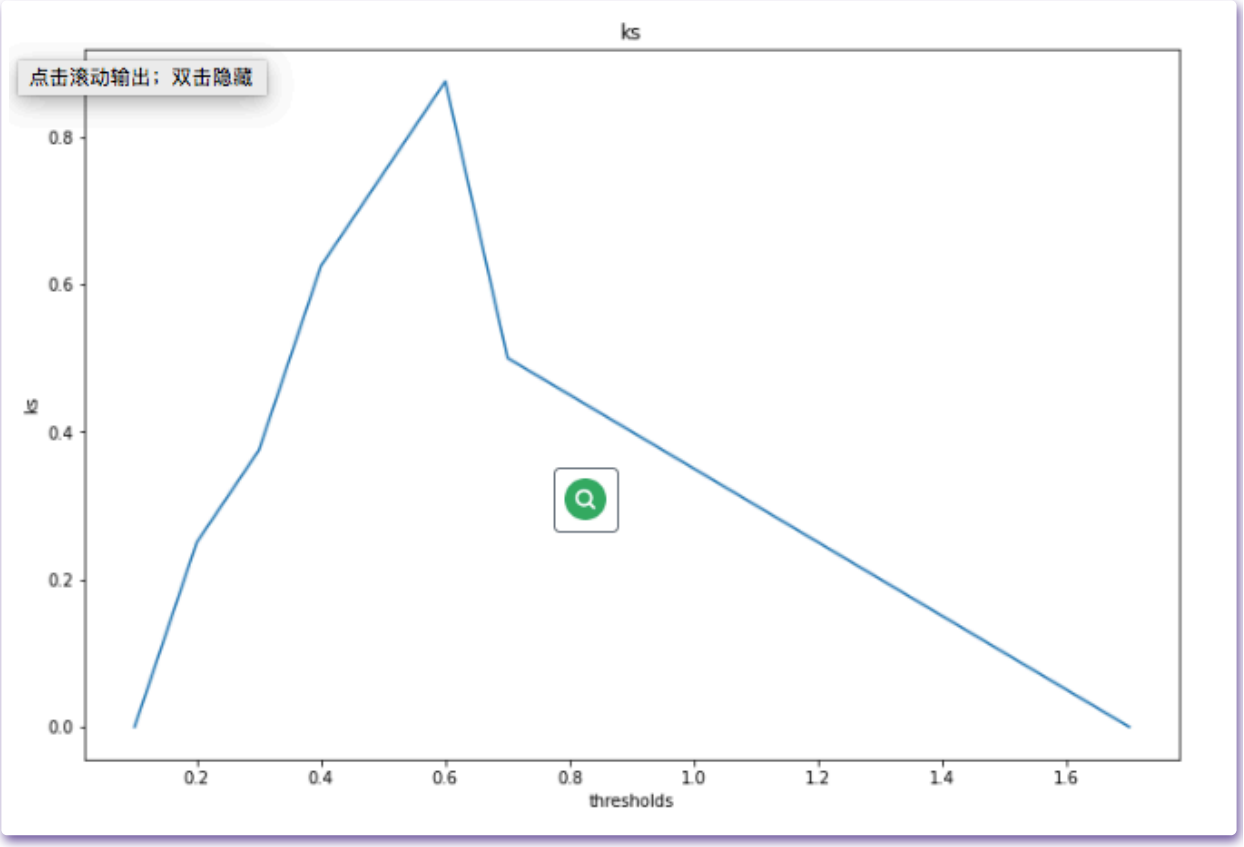
$$KS = \max(tpr, fpr) \tag{3}$$

```
"""2、ks = max(tpr-fpr)"""
fpr,tpr,thresholds = metrics.roc_curve(y_true,y_pred_prob)
ks2 = max(tpr-fpr)
ks2

"""输出"""
0.875
```

- ks曲线（ks曲线的横纵坐标，还可以是分桶桶号，以及好坏样本累计占比率等等；）

```
"""python绘制ks曲线"""
def ks_plot(y_true,y_pred_prob,title='ks'):
    """
    y_true:真实标签
    y_pred_prob:预测概率
```

```
    """
    fpr,tpr,thresholds = metrics.roc_curve(y_true,y_pred_prob,pos_label=1)
    ks=tpr-fpr

    plt.figure(figsize=(12,8))
#     plt.plot(thresholds,tpr)
#     plt.plot(thresholds,fpr)
    plt.plot(thresholds,ks)
    plt.xlabel('thresholds')
    plt.ylabel('ks')
    plt.title(title)
    plt.show()
    print('ks',max(ks)) # max(tpr-fpr)
    print('thresholds',thresholds[ks.argmax()])
    return max(ks),thresholds[ks.argmax()]

ks_plot(y_true,y_pred_prob)

"""输出"""
ks 0.875
thresholds 0.6
```



5.4 lift-提升度

- Lift衡量的是，与不利用模型相比，模型的预测能力“变好”了多少，结合二分类模型的混淆矩阵

	预测positive	预测negative
实际positive	TP	FN
实际negative	FP	TN

$$lift = \frac{precision}{\text{实际正样本率}} = \frac{\frac{TP}{TP+FP}}{\frac{TP+FN}{TP+TN+FN+FP}} \tag{4}$$

- 例子

```python
import pandas as pd
import numpy as np
from sklearn.metrics import precision_score as ps

y_true = np.array([1,1,0,0,0,0,0,0,0,0])
y_pred = np.array([1,1,0,0,0,1,1,0,0,0])

# 精准、查准、命中
precision = ps(y_true,y_pred) #0.5
# 真实正样本
pos_rate = y_true.sum()/y_true.shape[0] # 0.2

# 提升度
lift = precision / pos_rate
lift

"""输出"""
2.5
```

5.5 psi+csi

- **PSI与CSI之间的区别和联系**
    1. 群体稳定性指标（PSI）与特征稳定性指标（CSI）非常类似，都是在衡量稳定性。当然，PSI的名气远远大于CSI，以致于很多人都不清楚这一指标。两者的区别和联系大致可以总结如下：
    2. **群体稳定性报告**（Population Stability Report）是用**模型分数层**（score）来评估当前样本与开发样本之间的分布差异。目前业内有可以统一参考来判断稳定性的阈值标准。
    3. **特征稳定性报告**（Characteristic Stability Report）是从**入模特征层**（characteristic）来分析当前样本与开发样本之间的分数差异，以及对最终模型分数的影响。目前没有统一参考来判断稳定性的阈值标准。

5.5.1 PSI-群体稳定性指标(连续型)

- `PSI` (Population Stability Index:群体稳定性指标)

  在风控中，一套模型上线后往往需要很久（通常一年以上），如果模型不稳定会直接影响决策的合理性，所以稳定性压倒一切, `PSI` 反应了验证样本在各个分布与建模样本分布的稳定性，常用来筛选特征变量，评估模型稳定性

  > 入模变量保证稳定性，变量监控
  >
  > 模型分数保证稳定性，模型监控

  其中在建模时通常以

  1. 训练样本（In the Sample,INS）作为预期分布
  2. 验证样本作为实际分布，训练样本包括
     - 样本外(Out of Sample,OOS)
     - 跨时间样本(Out of Time,OOT)

- `PSI计算公式`：

$$
\begin{aligned}
PSI &= sum\{(\text{实际占比} - \text{预期占比}) * \ln(\frac{\text{实际占比}}{\text{预期占比}})\} \\
&= \sum_{buckets} (actual_pct - expect_pct) * \ln(\frac{actual_pct}{expect_pct})
\end{aligned} \tag{5}
$$

  其中， `origin_percent` 表示实际数据当前分箱样本数占比、 `new_percent` 表示预期数据当前分箱样本数占比。

- 注意： `np.log` 基数默认为 `e` ,信息论中尝尝选择2，因此信息的单位是 `比特` (`bits`),而机器学习中基数长选择为自然常数e，因此单位常被称为 `奈特` (`nats`)

- `PSI数值范围`

PSI范围	稳定性	建议事项
0 ~ 0.1	好	没有变化或者很少变化
0,1 ~ 0.25	略不稳定	有变化，继续监控后续变化
> 0.25	不稳定	发生大变化，进行特征项分析

- `代码`

```python
import pandas as pd
import numpy as np
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from tqdm import notebook

"""创建数据"""
cancer = load_breast_cancer()
df = pd.DataFrame(cancer.data,columns=['_'.join(i.split()) for i in cancer.feature_names])
```

```python
df['y']  = cancer.target

X_train,X_test,y_train,y_test = train_test_split(df.iloc[:,:-1],df['y'],test_size=.2)

print(X_train.shape) # (455,30)
print(X_test.shape)  # (114,30)
```

psi计算代码

```python
def psi_calculate(origin,new,feature_name,origin_y=None
                  ,y_name=None,buckets_type='cut',bins_num=10):
    """计算单个连续型变量的psi
        origin为实际数据,new为预期数据
        无监督分箱,当bucket_type 为 cut、qcut 时,无需借助目标变量
        有监督分箱,当bucket_type 为 tree、chimerge时,此时借助scorecardpy库

    Parameters
    ----------
    :param                origin:            DataFrame,实际数据
    :param                new:               DataFrame,预期数据
    :param                feature_name:      string,需要计算PSI的字段(类别型)
    :param                origin_y:          Series,y值
    :param                y_name:            string,目标变量名称
    :param                buckets_type:      string,分箱方式: cut、qcut、tree、chimerge
    :param                bins_num:          int,分箱数


    Returns
    ----------
    :return             psi:                  float,psi值
                        psi_df:               DataFrame,psi详细

    Examples
    ----------
    --等频、等距分箱(无监督分箱)
    >>> psi,psi_df = psi_calculate(origin=X_train
                                   ,new=X_test
                                   ,feature_name='mean_radius'
                                   ,buckets_type='qcut'
                                   ,bins_num=10
                                   )
    --决策树, 卡方分箱(有监督分箱)
    >>> psi,psi_df = psi_calculate(origin=X_train
                                   ,new=X_test
                                   ,feature_name='mean_radius'
```

```
                                    ,origin_y=y_train
                                    ,y_name='y'
                                    ,buckets_type='chimerge'
                                    )

    """
    origin = origin[[feature_name]]
    new = new[[feature_name]]

    if buckets_type == 'cut':    # 等宽分箱
        origin_min = origin[feature_name].min() # 最小值
        origin_max = origin[feature_name].max() # 最大值
        binlen     = (origin_max-origin_min) / bins_num #等频率每一箱长度
        bins       = [origin_min + i * binlen for i in range(1, bins_num)]#设定分组
        bins.insert(0, -float("inf"))
        bins.append(float("inf"))
        print(bins)
        origin_cut = pd.cut(origin[feature_name]
                            ,bins=bins
).value_counts(sort=False).reset_index()
        new_cut    = pd.cut(new[feature_name]
                            ,bins=bins
).value_counts(sort=False).reset_index()

        origin_cut.columns  = ['buckets','origin_cnt']
        new_cut.columns     = ['buckets','new_cnt']

    elif buckets_type == 'qcut': # 等频率分箱

        qcut_data = pd.qcut(origin[feature_name]
                            ,q=bins_num
                            ,duplicates='drop'
                            # ,retbins=True
                           )

        origin_cut = origin[feature_name].groupby(qcut_data).count().rename('origin_cnt')

        qcut_bins = origin_cut.index.categories # 等频分箱的bins,如果直接用retbins返回的会有浮点数误差
        origin_cut = origin_cut.reset_index()

        new_cut    = new[feature_name].groupby(pd.cut(new[feature_name]
,bins=qcut_bins
                                                      )
```

```
).count().rename('new_cnt').reset_index()

        origin_cut.columns  = ['buckets','origin_cnt']
        new_cut.columns     = ['buckets','new_cnt']
    elif buckets_type in ['tree','chimerge']: ## 借助scorecardpy 库的分箱,分箱方法为决策树或者卡方
        origin = origin[[feature_name]]
        new = new[[feature_name]]

        ## 卡方和决策树分箱都是有监督分箱，需要借助目标变量
        origin_cut_data = pd.concat([origin[feature_name],origin_y],axis=1)

        origin_cut = sc.woebin(origin_cut_data
                               ,y=y_name
                               ,method=buckets_type
                               )[feature_name]

        break_list = origin_cut.breaks.tolist()
        break_list = [float(i) for i in break_list]
        break_list.insert(0,-np.inf)

        origin_cut = origin_cut[['bin','count']]

        new_cut = new[feature_name].groupby(pd.cut(new[feature_name]
                                                   ,bins=break_list
                                                   ,right=False   # [left,right)
                                                   )
).count().rename('new_cnt').reset_index()

        new_cut[feature_name] = new_cut[feature_name].astype('str')
        new_cut[feature_name] = new_cut[feature_name].apply(lambda x: x.replace(' ',''))

        origin_cut.columns  = ['buckets','origin_cnt']
        new_cut.columns     = ['buckets','new_cnt']

        ## 解决为了防止浮点数误差导致的后续无法根据分割点merge
        ## [-inf,0.14999999999999997) [-inf,0.15)
        origin_cut['buckets'] = new_cut['buckets']

    else:
        print('bucket_types 只能在【cut、qcut、tree、chimerge】中')
        raise ValueError

    # print(origin_cut)
    origin_cut['feature'] = feature_name
```

```
    new_cut['feature']     = feature_name

    origin_cut = origin_cut[['feature','buckets','origin_cnt']]
    new_cut    = new_cut[['feature','buckets','new_cnt']]
    # print(origin_cut)
    # print(new_cut)
    psi_df = pd.merge(origin_cut,new_cut,on=['feature','buckets'])
    # print(psi_df)

    # 计算占比，分子加1，防止计算PSI时分子为0（这里分母不可能为0）
    psi_df['origin_percent'] = (psi_df['origin_cnt'] + 1) / psi_df['origin_cnt'].sum()
    psi_df['new_percent']    = (psi_df['new_cnt'] + 1) / psi_df['new_cnt'].sum()
    psi_df['minus']          = psi_df.apply(lambda x: x['origin_percent']-x['new_percent'],axis=1)
    psi_df['log']            = psi_df.apply(lambda x: np.log(x['origin_percent']/x['new_percent']),axis=1)


    # psi_df['psi'] = psi_df.apply(lambda x: (x['origin_percent']-x['new_percent']) *\
    # np.log(x['origin_percent']/x['new_percent'])
    #                                ,axis=1)

    psi_df['psi_bucket']  = psi_df.apply(lambda x: x['minus'] * x['log'],axis=1)
    psi_df['psi']         = psi_df['psi_bucket'].sum()

    psi = psi_df['psi_bucket'].sum()

    return psi,psi_df
```

计算单个变量psi（等频）

```
# 计算单个指标的psi
psi,psi_df = psi_calculate(X_train,X_test,buckets_type='qcut',feature_name='mean_radius')
psi

"""输出"""
0.11023008607241508
```

psi_df

executed in 24ms, finished 11:21:23 2023-03-10

	feature	buckets	origin_cnt	new_cnt	origin_percent	new_percent	minus	log	psi_bucket	psi
0	mean_radius	(6.9799999999999995, 10.368]	46	14	0.103297	0.131579	-0.028282	-0.242002	0.006844	0.11023
1	mean_radius	(10.368, 11.446]	45	15	0.101099	0.140351	-0.039252	-0.328046	0.012876	0.11023
2	mean_radius	(11.446, 12.06]	47	10	0.105495	0.096491	0.009003	0.089207	0.000803	0.11023
3	mean_radius	(12.06, 12.842]	44	18	0.098901	0.166667	-0.067766	-0.521875	0.035365	0.11023
4	mean_radius	(12.842, 13.48]	46	10	0.103297	0.096491	0.006805	0.068153	0.000464	0.11023
5	mean_radius	(13.48, 14.278]	45	12	0.101099	0.114035	-0.012936	-0.120407	0.001558	0.11023
6	mean_radius	(14.278, 15.116]	45	5	0.101099	0.052632	0.048467	0.652783	0.031639	0.11023
7	mean_radius	(15.116, 17.3]	46	14	0.103297	0.131579	-0.028282	-0.242002	0.006844	0.11023
8	mean_radius	(17.3, 19.59]	46	9	0.103297	0.087719	0.015577	0.163464	0.002546	0.11023
9	mean_radius	(19.59, 28.11]	45	7	0.101099	0.070175	0.030923	0.365101	0.011290	0.11023

计算单个变量psi（以卡方分箱为例子）

```
psi,psi_df = psi_calculate(origin=X_train
                           ,new=X_test
                           ,feature_name='mean_radius'
                           ,origin_y=y_train
                           ,y_name='y'
                           ,buckets_type='chimerge'
                           )
psi

"""输出"""
0.02009471667376726
```

psi_df

executed in 19ms, finished 11:26:55 2023-03-10

	feature	buckets	origin_cnt	new_cnt	origin_percent	new_percent	minus	log	psi_bucket	psi
0	mean_radius	[-inf,13.0)	199	57	0.439560	0.508772	-0.069211	-0.146225	0.010120	0.020095
1	mean_radius	[13.0,15.0)	113	26	0.250549	0.236842	0.013707	0.056263	0.000771	0.020095
2	mean_radius	[15.0,16.0)	29	4	0.065934	0.043860	0.022074	0.407660	0.008999	0.020095
3	mean_radius	[16.0,inf)	114	27	0.252747	0.245614	0.007133	0.028629	0.000204	0.020095

计算所有指标的psi（以卡方分箱为例子）

```
# 计算所有指标的psi
psi_list = []
psi_df_list = []
for feature in notebook.tqdm(X_train.columns):
    print(feature)
    psi,psi_df = psi_calculate(origin=X_train
                               ,new=X_test
                               ,feature_name=feature
                               ,origin_y=y_train
                               ,y_name='y'
                               ,buckets_type='chimerge'
```

```
                                )
        psi_list.append((feature,psi))
        psi_df_list.append(psi_df)

    psi = pd.DataFrame(psi_list,columns=['feature','psi'])
    psi_df = pd.concat(psi_df_list,ignore_index=False)
```

psi

executed in 15ms, finished 13:48:08 2023-03-10

	feature	psi
0	mean_radius	0.020095
1	mean_texture	0.034207
2	mean_perimeter	0.063369
3	mean_area	0.030114
4	mean_smoothness	0.079553
5	mean_compactness	0.035427
6	mean_concavity	0.037970
7	mean_concave_points	0.052567
8	mean_symmetry	0.021437
9	mean_fractal_dimension	0.011320
10	radius_error	0.064679

psi_df

executed in 27ms, finished 13:50:17 2023-03-10

	feature	buckets	origin_cnt	new_cnt	origin_percent	new_percent	minus	log	psi_bucket	psi
0	mean_radius	[-inf,13.0)	199	57	0.439560	0.508772	-0.069211	-0.146225	0.010120	0.020095
1	mean_radius	[13.0,15.0)	113	26	0.250549	0.236842	0.013707	0.056263	0.000771	0.020095
2	mean_radius	[15.0,16.0)	29	4	0.065934	0.043860	0.022074	0.407660	0.008999	0.020095
3	mean_radius	[16.0,inf)	114	27	0.252747	0.245614	0.007133	0.028629	0.000204	0.020095
0	mean_texture	[-inf,15.0)	76	15	0.169231	0.140351	0.028880	0.187118	0.005404	0.034207
...	...	...	...	...	...	...	...	...	...	...
2	worst_symmetry	[0.34,0.36)	25	1	0.057143	0.017544	0.039599	1.180850	0.046760	0.052286
3	worst_symmetry	[0.36,inf)	45	12	0.101099	0.114035	-0.012936	-0.120407	0.001558	0.052286
0	worst_fractal_dimension	[-inf,0.09)	320	86	0.705495	0.763158	-0.057663	-0.078566	0.004530	0.013771
1	worst_fractal_dimension	[0.09,0.1)	62	11	0.138462	0.105263	0.033198	0.274129	0.009101	0.013771
2	worst_fractal_dimension	[0.1,inf)	73	17	0.162637	0.157895	0.004743	0.029594	0.000140	0.013771

126 rows × 10 columns

5.5.2 PSI-群体稳定性指标(离散型)

- 代码

```
import pandas as pd

df = pd.read_csv('/Users/mengzhichao/Desktop/文件/图表/小微企业用电量数据.csv')

X_train = df.sample(n=7000)
X_test = df.sample(n=3000)
```

- 计算单个psi

```
def psi_calculate_category(origin,new,feature_name):
    """计算单个类别型变量的psi
       origin为实际数据，new为预期数据

    Parameters
    ----------------------------------------------------------------------
    :param                origin                DataFrame,实际数据
    :param                new                   DataFrame,预期数据
    :param                feature_name          string,需要计算PSI的字段(类别型)

    Returns
    ----------------------------------------------------------------------
                          psi:                   float,psi值
                          psi_df:                DataFrame,psi详细
    Examples
    ----------------------------------------------------------------------
    >>> psi,psi_df = psi_calculate_category(origin=X_train
                                            ,new=X_test
                                            ,feature_name='dimvl_name')

    """
    origin_cut = origin[feature_name].value_counts(dropna=False).reset_index()
    origin_cut.columns = ['buckets','origin_cnt']
    origin_cut['feature'] = feature_name
    origin_cut = origin_cut[['feature','buckets','origin_cnt']]

    new_cut = new[feature_name].value_counts(dropna=False).reset_index()
    new_cut.columns = ['buckets','new_cnt']
    new_cut['feature'] = feature_name
```

```python
    new_cut = new_cut[['feature','buckets','new_cnt']]

    psi_df = pd.merge(origin_cut,new_cut,on=['feature','buckets'])
    # print(psi_df)

    # 计算占比，分子加1，防止计算PSI时分子为0（这里分母不可能为0）
    psi_df['origin_percent'] = (psi_df['origin_cnt'] + 1) / psi_df['origin_cnt'].sum()
    psi_df['new_percent']    = (psi_df['new_cnt'] + 1) / psi_df['new_cnt'].sum()
    psi_df['minus']          = psi_df.apply(lambda x: x['origin_percent']-x['new_percent'],axis=1)
    psi_df['log']            = psi_df.apply(lambda x: np.log(x['origin_percent']/x['new_percent']),axis=1)

    psi_df['psi_bucket']  = psi_df.apply(lambda x: x['minus'] * x['log'],axis=1)
    psi_df['psi']         = psi_df['psi_bucket'].sum()

    psi = psi_df['psi_bucket'].sum()

    return psi,psi_df

psi,psi_df = psi,psi_df = psi_calculate_category(origin=X_train
                                                 ,new=X_test
                                                 ,feature_name='区域')
```

psi

executed in 7ms, finished 17:17:15 2

```
0.0018855713990353201
```

psi_df

executed in 21ms, finished 17:17:07 2023-03-09

	feature	buckets	origin_cnt	new_cnt	origin_percent	new_percent	minus	log	psi_bucket	psi
0	区域	兰州新区	818	342	0.117000	0.114333	0.002667	0.023056	0.000061	0.001886
1	区域	榆中县	791	354	0.113143	0.118333	-0.005190	-0.044854	0.000233	0.001886
2	区域	安宁区	783	328	0.112000	0.109667	0.002333	0.021053	0.000049	0.001886
3	区域	西固区	779	320	0.111429	0.107000	0.004429	0.040555	0.000180	0.001886
4	区域	城关区	778	316	0.111286	0.105667	0.005619	0.051811	0.000291	0.001886
5	区域	皋兰县	776	332	0.111000	0.111000	0.000000	0.000000	0.000000	0.001886
6	区域	红古区	774	329	0.110714	0.110000	0.000714	0.006473	0.000005	0.001886
7	区域	七里河区	752	326	0.107571	0.109000	-0.001429	-0.013193	0.000019	0.001886
8	区域	永登县	749	353	0.107143	0.118000	-0.010857	-0.096522	0.001048	0.001886

计算所有类别型：

```python
## 所有的类别型特征列表
all_category_feature_list = df.select_dtypes(include='object').columns.tolist()

psi_list = []
psi_df_list = []
for feature in notebook.tqdm(all_category_feature_list):
    psi,psi_df = psi_calculate_category(X_train,X_test,feature_name=feature)
    psi_list.append((feature,psi))
    psi_df_list.append(psi_df)

psi = pd.DataFrame(psi_list,columns=['feature','psi'])
psi_df = pd.concat(psi_df_list,ignore_index=False)
```

psi

executed in 14ms, finished 17:29:18 2023-03-09

	feature	psi
0	日期	3.015927e-01
1	城市	3.627254e-08
2	区域	1.885571e-03
3	分类	2.135011e-03
4	等级	5.972658e-04

psi_df

executed in 29ms, finished 17:29:41 2023-03-09

	feature	buckets	origin_cnt	new_cnt	origin_percent	new_percent	minus	log	psi_bucket	psi
0	日期	2019-04-01	9	2	0.001642	0.001005	0.000637	0.490924	3.127243e-04	0.301593
1	日期	2019-08-24	9	2	0.001642	0.001005	0.000637	0.490924	3.127243e-04	0.301593
2	日期	2017-05-06	8	3	0.001478	0.001340	0.000138	0.097882	1.348803e-05	0.301593
3	日期	2018-10-12	8	1	0.001478	0.000670	0.000808	0.791029	6.390057e-04	0.301593
4	日期	2018-10-30	8	2	0.001478	0.001005	0.000473	0.385564	1.822975e-04	0.301593
...	...	...	...	...	...	...	...	...	...	...
0	等级	NaN	5898	2524	0.842714	0.841667	0.001048	0.001244	1.303156e-06	0.000597
1	等级	黄	784	337	0.112143	0.112667	-0.000524	-0.004660	2.440972e-06	0.000597
2	等级	蓝	195	83	0.028000	0.028000	0.000000	0.000000	0.000000e+00	0.000597
3	等级	橙	100	42	0.014429	0.014333	0.000095	0.006623	6.307182e-07	0.000597
4	等级	红	23	14	0.003429	0.005000	-0.001571	-0.377294	5.928909e-04	0.000597

1417 rows × 10 columns

5.5.3 CSI-特征稳定性指标

- csi公式

$$CSI = \sum_{buckets=1}^{n} (actual_pct - expect_pct) * partial_score \tag{6}$$

- 代码

```python
import pandas as pd
import numpy as np

# 假设score_data是一个经过woe分箱和评分卡转换的分数表
score_data = pd.read_csv('./need/score_data_buckets_partial_score.csv')

score_data['csi_bucket'] = (score_data['actual_pct']-score_data['expect_pct'])*score_data['partial_score']

score_data
```

	feature	base_score	buckets	partial_score	total	actual	expect	actual_pct	expect_pct	csi_bucket
0	age	482.011128	(-inf, 36.0]	-4.111495	39638	24841	14797	0.255259	0.151467	-0.426737
1	age	482.011128	(36.0, 54.0]	-2.200319	90531	51461	39070	0.528798	0.399934	-0.283540
2	age	482.011128	(54.0, 61.0]	1.971214	27954	12211	15743	0.125477	0.161151	-0.070322
3	age	482.011128	(61.0, 74.0]	7.921794	27758	7414	20344	0.076184	0.208248	-1.046187
4	age	482.011128	(74.0, inf]	13.493742	9127	1390	7737	0.014283	0.079199	-0.875953
5	RevolvingUtilizationOfUnsecuredLines	482.011128	(-inf, 0.099]	47.574785	48755	4852	43903	0.049858	0.449407	-19.008463
6	RevolvingUtilizationOfUnsecuredLines	482.011128	(0.099, 0.298]	14.506618	29270	9884	19386	0.101565	0.198442	-1.405358
7	RevolvingUtilizationOfUnsecuredLines	482.011128	(0.298, 0.465]	-2.800291	19486	10354	9132	0.106395	0.093478	-0.036169
8	RevolvingUtilizationOfUnsecuredLines	482.011128	(0.465, 0.982]	-23.219321	68165	50742	17423	0.521409	0.178348	-7.965652
9	RevolvingUtilizationOfUnsecuredLines	482.011128	(0.982, 1.0]	-10.225833	16624	10238	6386	0.105203	0.065369	-0.407328
10	RevolvingUtilizationOfUnsecuredLines	482.011128	(1.0, inf]	-44.102769	12708	11247	1461	0.115571	0.014955	-4.437420
11	DebtRatio	482.011128	(-inf, 0.0175]	33.889402	9761	1750	8011	0.017982	0.082003	-2.169633
12	DebtRatio	482.011128	(0.0175, 0.402]	0.806052	87817	43038	44779	0.442245	0.458374	-0.013000
13	DebtRatio	482.011128	(0.402, 1.467]	-8.695052	58430	34782	23648	0.357409	0.242069	-1.002887
14	DebtRatio	482.011128	(1.467, inf]	3.939728	39000	17747	21253	0.182363	0.217553	-0.138641

```python
score_data.groupby('feature',as_index=False)['csi_bucket'].sum()
```

	feature	csi_bucket
0	DebtRatio	-3.324161
1	MonthlyIncome	-1.670661
2	NumberOfDependents	-7.709427
3	NumberOfOpenCreditLinesAndLoans	-0.614209
4	NumberOfTime30-59DaysPastDueNotWorse	-6.025813
5	NumberOfTime60-89DaysPastDueNotWorse	-1.469217
6	NumberOfTimes90DaysLate	-7.538880
7	NumberRealEstateLoansOrLines	-4.092454
8	RevolvingUtilizationOfUnsecuredLines	-33.260390
9	age	-2.702739

5.6 KL散度

- KL散度的概念来源于概率论和信息论中。KL散度又被称为：`相对熵`、互熵、鉴别信息、Kullback熵、Kullback-Leible散度(即KL散度的简写)。在机器学习、深度学习领域中，KL散度被广泛运用于变分自编码器中(Variational AutoEncoder,简称VAE)、EM算法、GAN网络中
- KL散度的定义是建立在熵(Entropy)的基础上的。此处以离散随机变量为例，先给出`熵的定义`，再给定KL散度定义。

5.6.1 信息熵

5.6.1.1 公式

- 一个离散随机变量X的可能取值为$X = x_1, x_2, \ldots, x_n$，而对应的概率为$p_i = p(X = x_i)$，如下

x_1	x_2	x_3	x_4	……	x_n
p(x_1)	p(x_2)	p(x_3)	p(x_4)	……	p(x_n)

在信息论中，某个信息 x_i 出现的不确定性的大小定义为 x_i 所携带的信息量，用 $I(x_i)$ 表示。$I(x_i)$ 与信息 x_i 出现的概率 $p(x_i)$ 之间的关系为

$$\begin{aligned} I(x_i) &= \log \frac{1}{p(x_i)} \\ &= -\log p(X_i) \end{aligned} \tag{7}$$

以上是求单一信息的信息量；求全部信息的平均信息量，即

1. `离散型随机变量的信息熵`定义为：

$$
\begin{aligned}
H(x) &= -\sum_{i=1}^{n} p(x_i)\log p(x_i) \\
&= H(P)
\end{aligned} \tag{8}
$$

2. 连续型随机变量的信息熵 定义为：

$$
H(x) = -\int f(x)\log(f(x))dx \tag{9}
$$

规定当 $p(x_i)=0$ 时，$p(x_i)\log p(x_i)=0$。

- 信息熵 是一个平均信息量，可以解释为：用基于P的编码去编码来自P的样本，其最优编码平均所需要的比特个数。

5.6.1.2 例子

- 以《数学之美》中一个小例子来理解上述的公式，世界杯有32支球队，赛后我问一个知道比赛的观众“哪支球队是冠军”？他不愿告诉我，让我猜，并且没猜一次，他需要收一块钱(一块钱能买1G流量,上网查它不香吗？)才肯告诉我是否才对，我可以吧球队编号从1到32，然后提问“冠军是在1-16号球队中吗？”假如他告诉我猜错了，那么肯定就在17-32号球队中，这样只需要提问5次($2^5=32$)，就能知道哪只球队是冠军。所以谁是世界杯冠军这条消息只值5元($\log_2^{32}=5$)。

  香农用“比特”(Bit)来度量信息量，一个比特是一位二进制数，在计算机中，一个字节是8比特。则在上面的例子中，这条信息量是5比特，信息量的比特数和所有可能情况的对数函数有关。即信息量

$$
\begin{aligned}
I(x_i) &= \log_2 \frac{1}{p(x_i)} \\
&= -\log_2 p(X_i) \\
&= \log_2 \frac{1}{\frac{1}{32}} \\
&= -\log_2 \frac{1}{32} \\
&= 5(bit)
\end{aligned} \tag{10}
$$

  当然我们发现实际上可能并不需要5次才能猜中，因为巴西，德国，意大利这样的球队会比其他球队更有可能夺冠，所以第一次猜测时不需要等分，而可以将少数热门球队分为一份，其他的另分为一份，重复这个过程，有可能3次或者4次就能猜出结果。所以，当没只球队夺冠的可能性(概率)不等时(使用$p_1,p_2,...,p_{32}$分别表示这32只球队夺冠的概率)，“谁是世界杯冠军”的信息量比5比特少。香农指出，它准确的信息量是

$$
\begin{aligned}
H(x) &= -\sum_{i=1}^{n} p(x_i)\log p(x_i) \\
&= H(P) \\
&= -(p_1\cdot\log p_1 + p_2\cdot\log p_2 + \cdots + p_{32}\cdot\log p_{32})
\end{aligned} \tag{11}
$$

  香农称之为信息熵，一般用H表示，单位是比特。当这32支球队夺冠概率相同时，对应的信息熵等于5比特。

- 汉字编码

  有一本50万字的书，假设每个字出现的概率相同，那么每个字携带的信息量是

$$\log_2^{500000} \approx 18.93bit \tag{12}$$

  即加入每个字等概率，大约需要19比特（19位二进制数）表示一个汉字

  这本书总携带的信息量为

$$\begin{aligned} 500000 * \log_2^{500000} &\approx 500000 * 18.93\text{Bit} \\ &= 9465000\text{Bit} \\ &= (9465000/8)\text{Byte} \\ &= 1183125\text{Byte} \\ &= (1183125/1024/1024)\text{G} \\ &\approx 1.13\text{G} \end{aligned} \tag{13}$$

  即在假设每个字出现的概率相同的情况下，50万字的书携带的信息量为9465000比特；

  但是汉子使用频率不均等，10%的汉字占常用文本的95%以上。且在考虑上下文的情况下，每个汉字的信息熵只有5bit左右，即50万字携带250万比特的信息，采用较好的算法压缩，整本书可以存成一个320k的文件，若直接用两字节的国标编码存储这本书，大约需要1MB大小，是压缩文件的3倍，这两个数据量的差异在信息论中叫冗余度，且250万比特在这只是一个平均数，同样两本50万字的书，所含的信息量可以相差很多。如果一本书重复的内容很多，它的信息量就小，冗余度就大。且不同语言的冗余度差别也很大，一本很厚的英文书翻译成中译本一般会薄很多。

5.6.1.3 熵的性质

- 对于离散型随机变量，当其服从均匀分布时，熵有极大值；
- 对与离散型随机变量，取某一个值的概率为1，其他所有值的概率为0时，熵有极小值。

5.6.2 联合熵

- 联合熵（Joint Entropy）是熵对多维概率分布的推广，它描述了一组随机变量的不确定性。以二维随机向量为例，有两个离散型随机变量X和Y，它们的联合概率质量函数为p(x)，联合熵定义为：

$$H(X,Y) = -\sum_x \sum_y p(x,y) \log p(x,y) \tag{14}$$

  推广到多个随机变量，有：

$$H(X_1, X_2, \ldots, X_n) = -\sum_x \sum_y p(x_1, x_2, \ldots, x_n) \log p(x_1, x_2, \ldots, x_n) \tag{15}$$

  对于连续型随机向量（X，Y），假设联合概率密度函数为p(x,y)，其联合熵为二重积分

$$H(X,Y) = -\int_{-\infty}^{+\infty}\int_{-\infty}^{+\infty} p(x,y)\log p(x,y)dxdy \tag{16}$$

即

1. 离散型随机变量的联合熵为

$$H(X,Y) = -\sum_x\sum_y p(x,y)\log p(x,y) \tag{17}$$

2. 连续型随机变量的联合熵为

$$H(X,Y) = -\int_{-\infty}^{+\infty}\int_{-\infty}^{+\infty} p(x,y)\log p(x,y)dxdy \tag{18}$$

5.6.3 交叉熵

5.6.3.1 定义公式

- 针对离散变量x的概率分布 $p(x)$, $q(x)$

x_1	x_2	x_3	x_4	……	x_n
p(x_1)	p(x_2)	p(x_3)	p(x_4)	……	p(x_n)
q(x_1)	q(x_2)	q(x_3)	q(x_4)	……	q(x_5)

其 交叉熵 定义为：

$$\begin{aligned} H(P,Q) &= \sum_{i=1}^{n} p(x_i)\log\frac{1}{q(x_i)} \\ &= -\sum_{i=1}^{n} p(x_i)\log q(x_i) \end{aligned} \tag{19}$$

- 交叉熵 可解释为：用基于p的编码去编码来自Q的样本，所需要的比特个数

5.6.3.2 性质

- 如果连个概率分布完全相等，交叉熵退化为熵
- 交叉熵不是距离，不具有对称性，也不满足三角不等式
- 当两个概率分布相等时，交叉熵有极小值
- 交叉熵与相对熵的关系，都反映了两个概率分布的差异程度。

5.6.4 KL散度（相对熵）

- KL散度或相对熵可通过下式得出

$$
\begin{aligned}
D_{KL}(P||Q) &= H(P,Q) - H(P) \\
&= -\sum_{i=1}^{n} p(x_i)\log q(x_i) - (-\sum_{i=1}^{n} p(x_i)\log p(x_i)) \\
&= \sum_{i=1}^{n} p(x_i)\log p(x_i) - \sum_{i=1}^{n} p(x_i)\log q(x_i) \\
&= \sum_{i=1}^{n} p(x_i)\left[\log p(x_i) + \log(\frac{1}{q(x_i)})\right] \\
&= \sum_{i=1}^{n} p(x_i)\log\frac{p(x_i)}{q(x_i)}
\end{aligned} \tag{20}
$$

- KL散度公式
  1. 离散型随机变量(P,Q为离散型)：

$$
D_{KL}(P||Q) = \sum_{i=1}^{n} p(x_i)\log\frac{p(x_i)}{q(x_i)} \tag{21}
$$

  2. 连续型随机变量(P,Q为离散型)：

$$
D_{KL}(P||Q) = \int_X P(x_i) * \left[\log\frac{p(x_i)}{q(x_i)}\right]dx \tag{22}
$$

- KL散度的意义：

  在统计学意义上来说，KL散度可以用来衡量两个分布之间的差异程度。若两者差异越小，KL散度越小，反之亦反。当两分布一致时，其KL散度为0。正是因为其可以衡量两个分布之间的差异，所以在VAE、EM、GAN中均有使用到KL散度。

5.6.5 交叉熵误差函数

- 举个例子，在神经网络中，对于 4 分类问题，如果某个 样本的真实标签是 4 类，那么标签的 One-hot 编码为[0,0,0,1]，即这张图片的分类是唯一 确定的，它属于第 4 类的概率$P(y$为$4|\boldsymbol{x}) = 1$，不确定性为 0，它的熵可以简单的计算为:

$$
-0 \bullet \log 0 - 0 \bullet \log 0 - 0 \bullet \log 0 - 1 \bullet \log 1 = 0 \tag{23}
$$

  也就是说，对于确定的分布，熵为 0，不确定性最低。

  由于$P(i) \in [0,1]$，$\log 2\ P(i) \le 0$，因此熵$H(P)$总是大于等于 0。当熵取得最小值 0 时， 不确定性为 0。分类问题的 One-hot 编码的分布就是熵为 0 的典型例子。

- 当分类问题中 y 的编码分布 p采用 One-hot 编码$\boldsymbol{y}$时:$H(p) = 0$，此时

$$
H(p,q) = H(p) + D_{KL}(p||q) = D_{KL}(p||q) \tag{24}
$$

  退化到真实标签分布$\boldsymbol{y}$与输出概率分布$\boldsymbol{o}$之间的 KL 散度上。

  则根据KL散度的定义，我们推导分类问题中交叉熵的计算表达式:

$$
\begin{aligned}
H(p,q) &= D_{KL}(p||q) \\
&= \sum y_j \log(\frac{y_j}{o_j}) \\
&= 1 * \log \frac{1}{o_i} + \sum_{j \neq i} 0 * \log \frac{0}{o_j} \\
&= -\log o_i
\end{aligned} \tag{25}
$$

其中i为 One-hot 编码中为 1 的索引号，也是当前输入的真实类别。可以看到，损失函数只与真实 类别i上的概率oi有关，对应概率oi越大，$H(p,q)$越小。当对应类别上的概率为 1 时，交叉 熵$H(p,q)$取得最小值 0，此时网络输出$\boldsymbol{o}$与真实标签$\boldsymbol{y}$完全一致，神经网络取得最优状态。

因此最小化交叉熵损失函数的过程也是最大化正确类别的预测概率的过程 。从这个角 度去理解交叉熵损失函数，非常地直观易懂。

6 补充

- 列表筛选元素

```
[i for i in df.columns if i not in ['str1','str2',...]]
```